Design and Analysis of Computer Algorithms

01:198:344

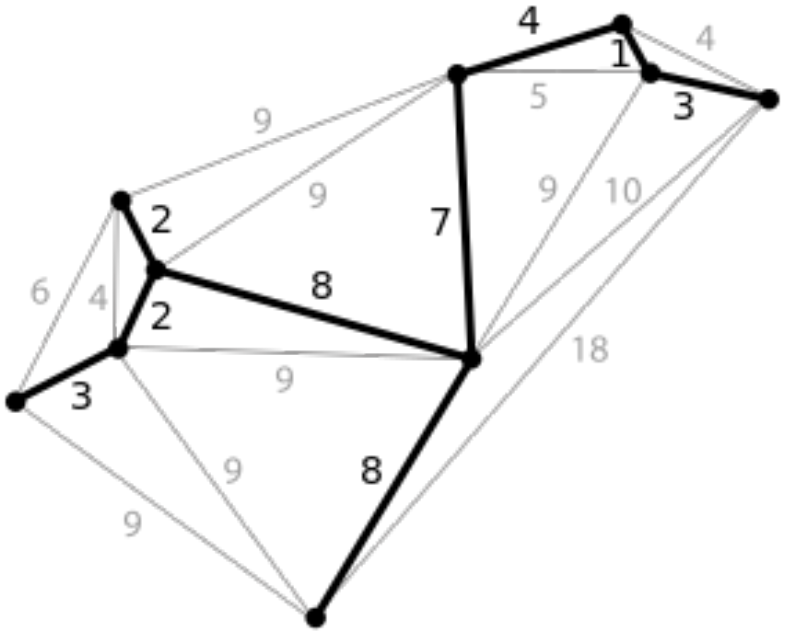
Spring 2026, Rutgers

Instructor: Surya Teja Gavva

Minimum Spanning Tree (MST)

For a connected, undirected, weighted graph $G = (V, E)$, a **Minimum Spanning Tree (MST)** is a subgraph $T \subset E$ that: Connects all vertices (spanning), forming a tree and has the minimum total edge weight among all spanning trees of G.

Weight of MST: $w(T) = \sum_{e \in T} w(e)$, where $w(e)$ is the weight of edge e.



Uniqueness of MST with Distinct Edge Weights

If all edge weights in a connected graph G are distinct, then G has a unique minimum spanning tree (MST).

Proof Outline: Assume there are two distinct MSTs, T and T'. Let e be the minimum-weight edge in $T \setminus T'$ and e' in $T' \setminus T$, with $w(e) \leq w(e')$. Adding e to T' forms a cycle C, which includes an edge $e'' \notin T$. Construct $T'' = T' + e - e''$. Since T' is an MST, $w(T'') = w(T')$, implying $w(e) = w(e'')$.

Generic MST Algorithm

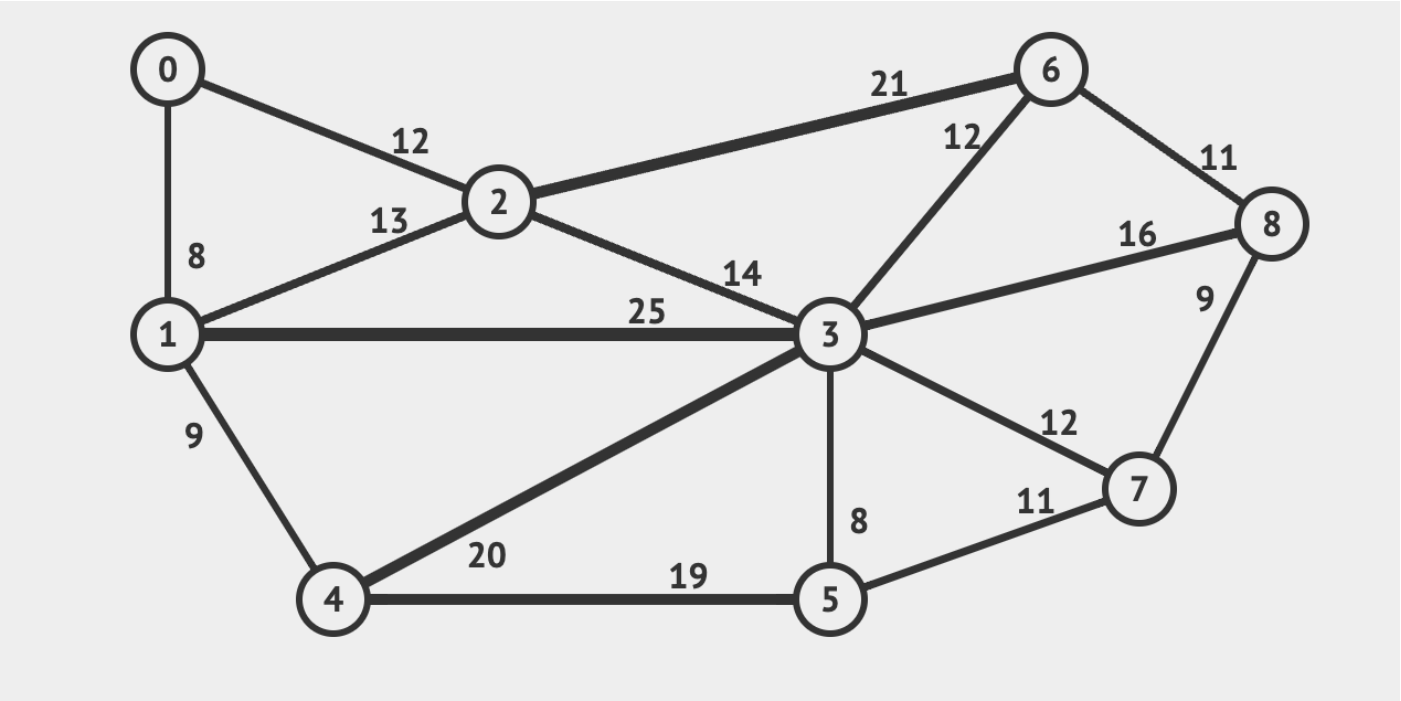
- The MST algorithm maintains a growing forest F of the input graph G.
- **Invariant**: F is always a subgraph of the MST of G.
- **Procedure**:
  - Start with each vertex as an isolated tree in F.
  - An edge is safe if it is the minimum-weight edge with exactly one endpoint in some component of F. Iteratively add the "safe" edge to F.
- When F becomes a single tree spanning all vertices, it is the MST.

Kruskal's Algorithm for MST

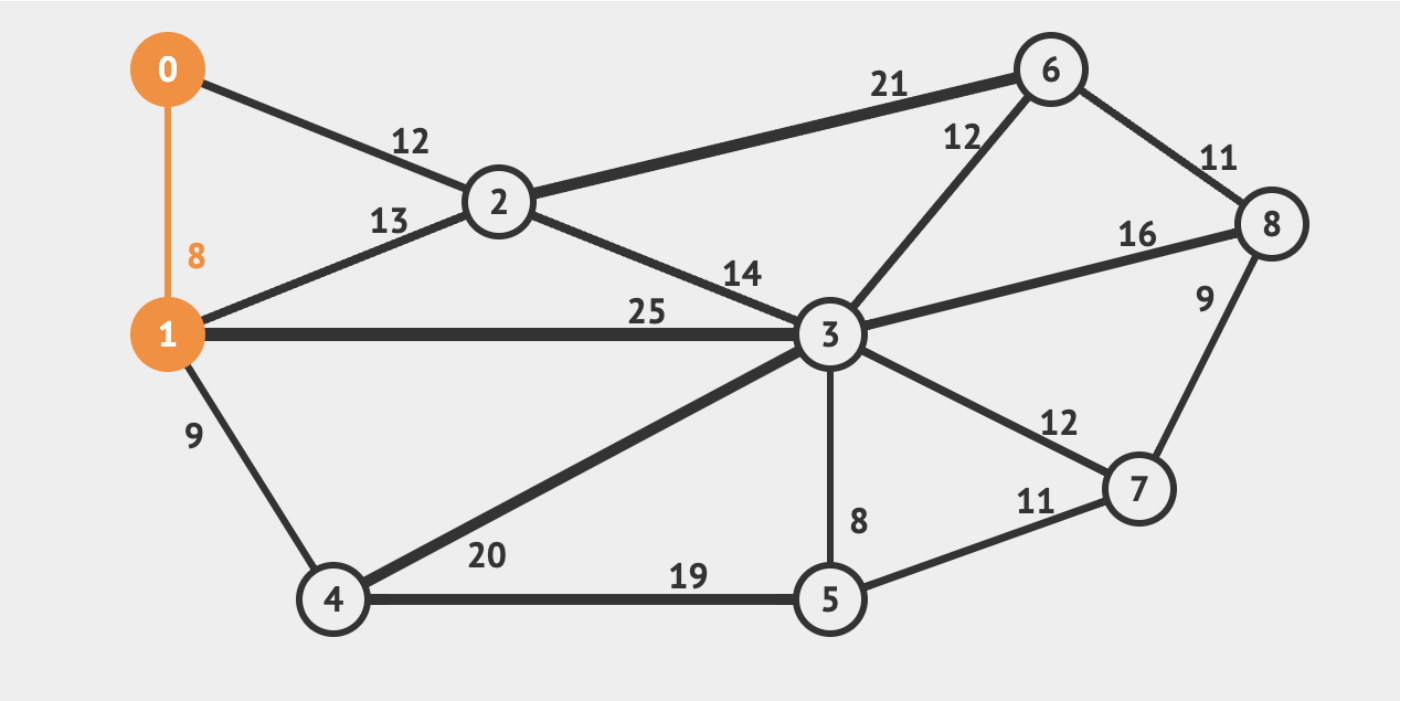
"Choose the cheapest useful edge possible"

1. Sort edges of G in increasing order by weight.
2. Initialize an empty forest $F = \emptyset$.
3. For each edge e in sorted order:
   - If e does not form a cycle in F, add e to F.
4. Return F as the MST.

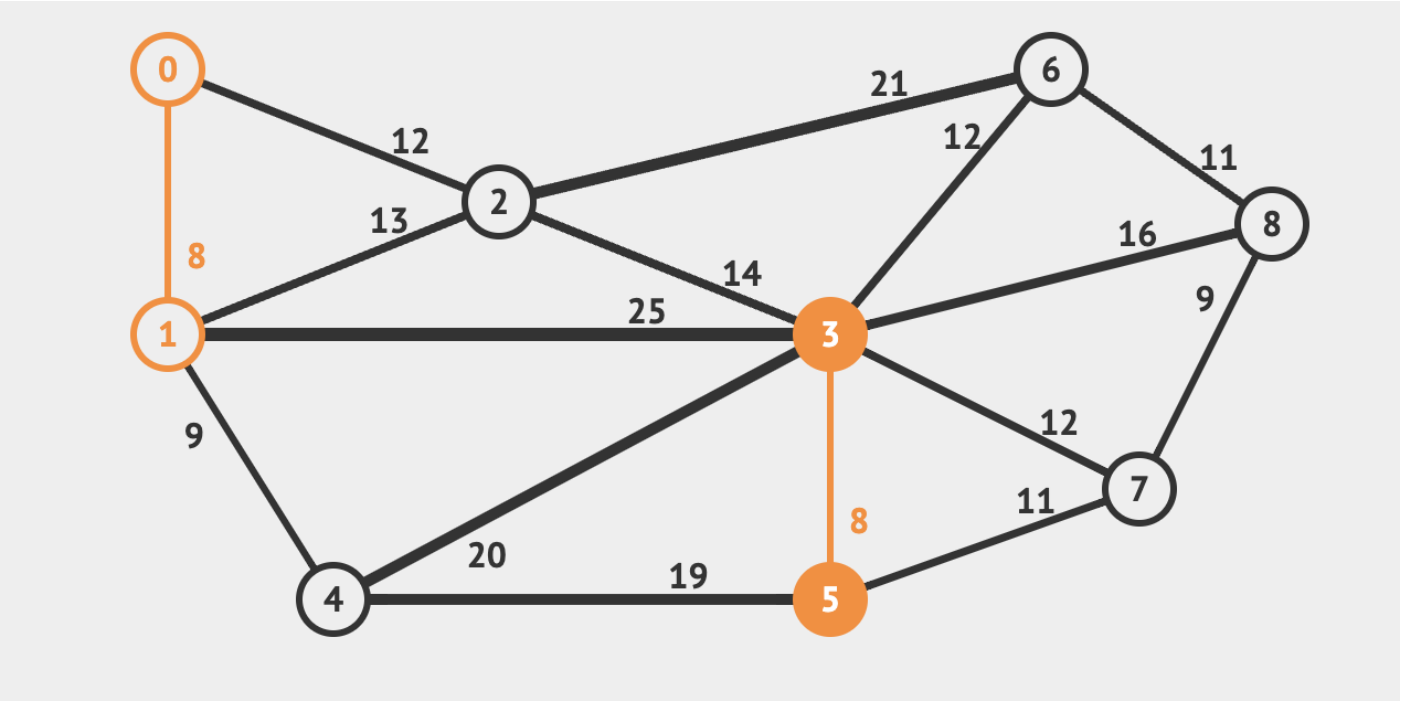
Kruskal's Algorithm Example



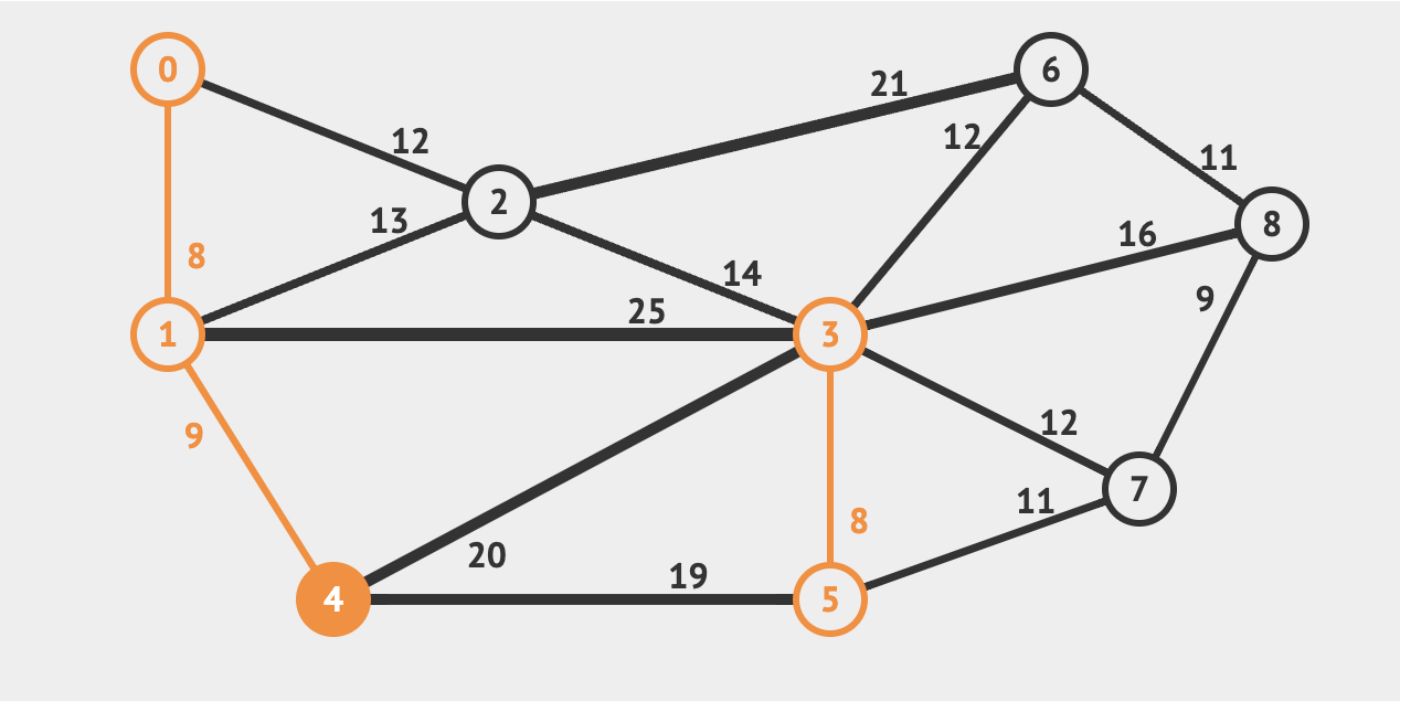
Kruskal's Algorithm Example



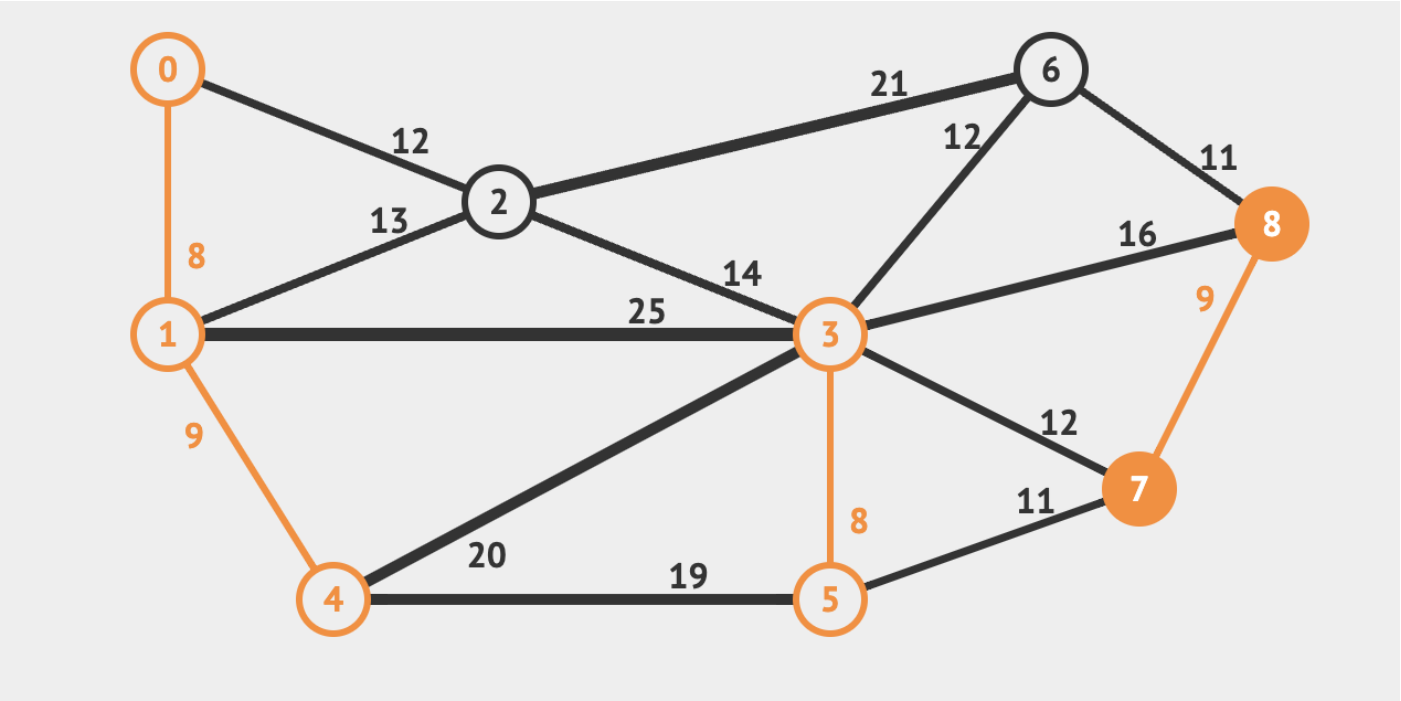
Kruskal's Algorithm Example



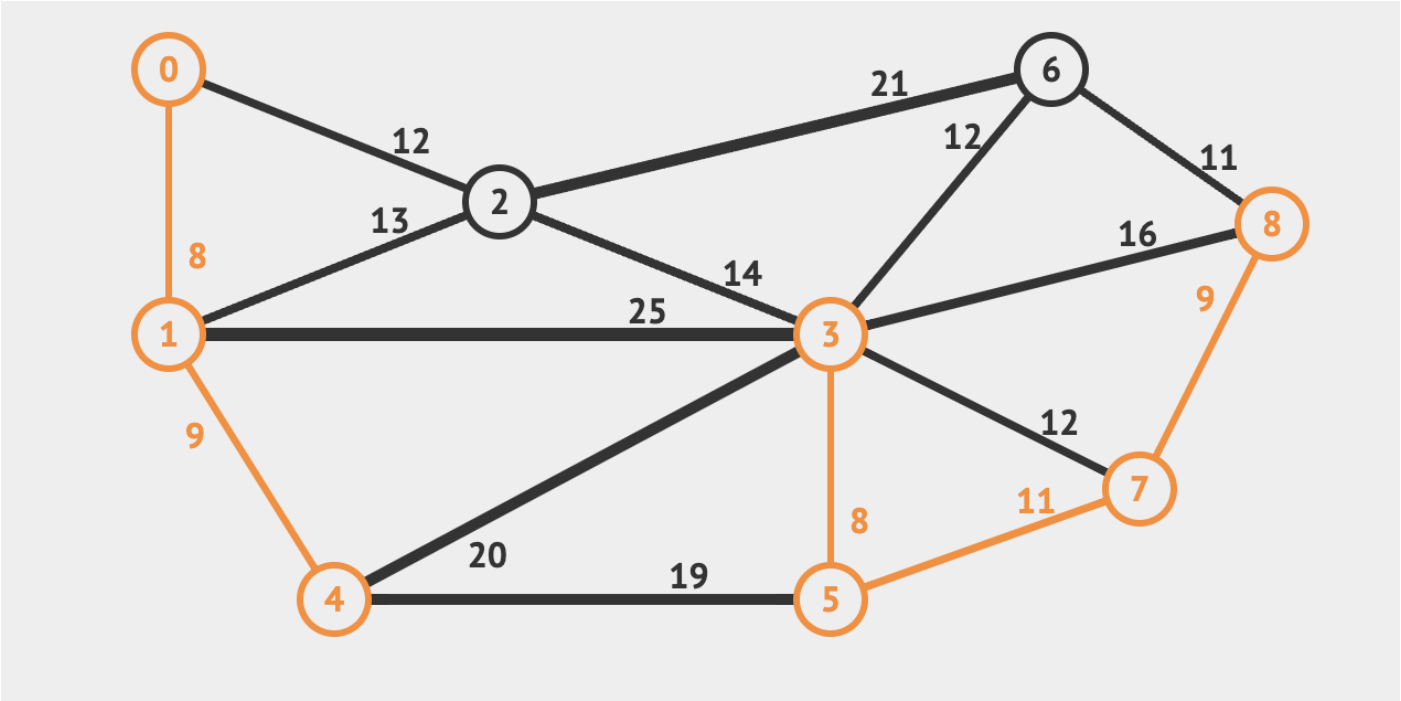
Kruskal's Algorithm Example



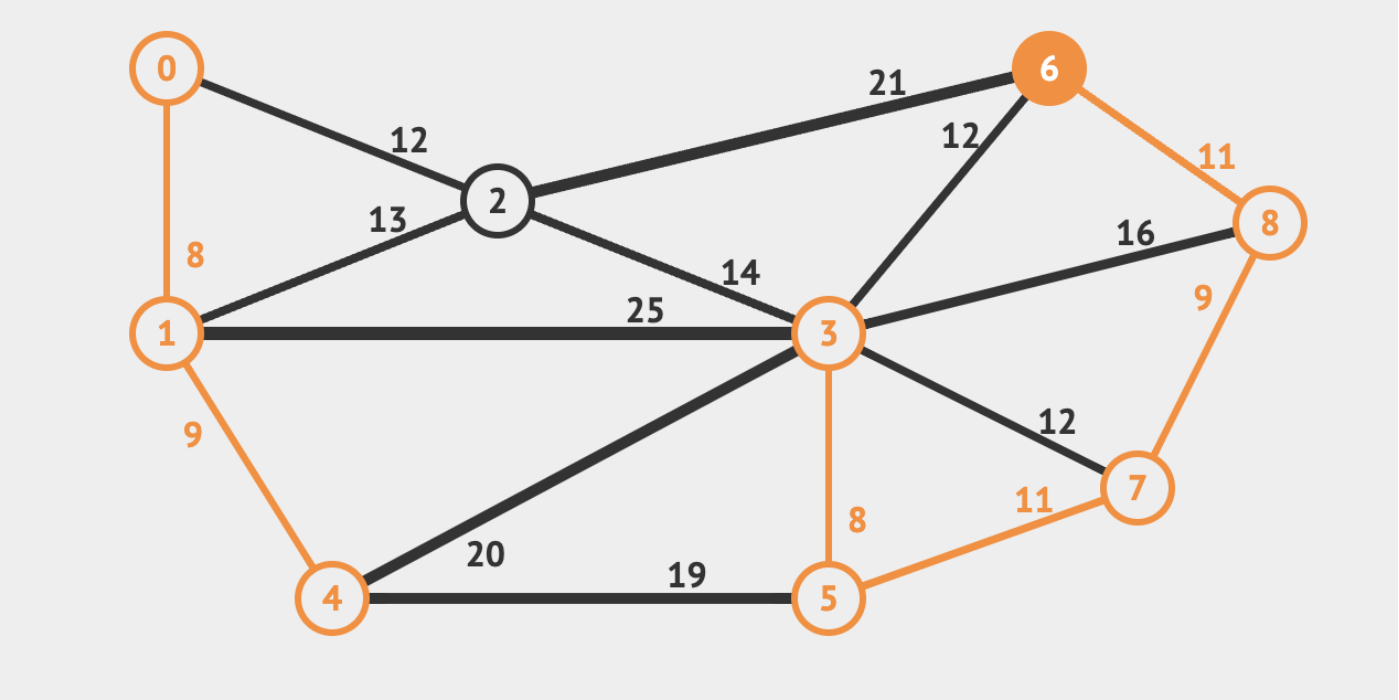
Kruskal's Algorithm Example



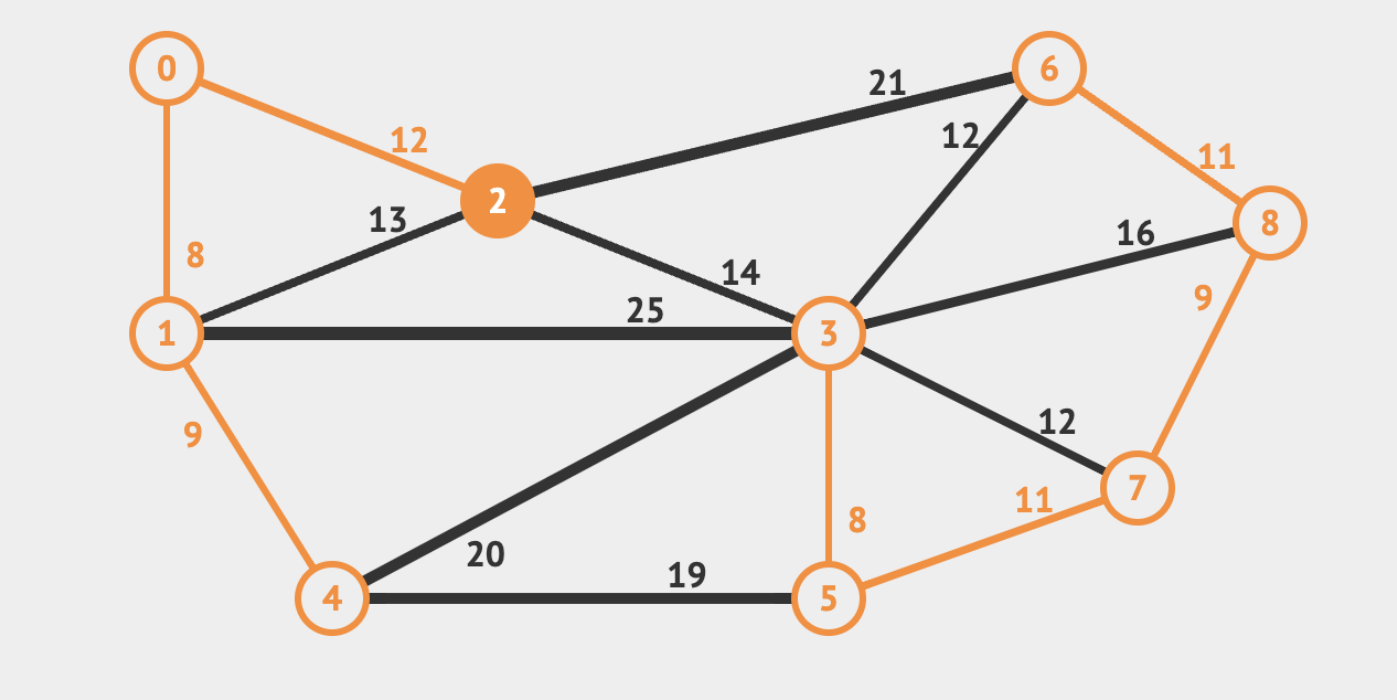
Kruskal's Algorithm Example



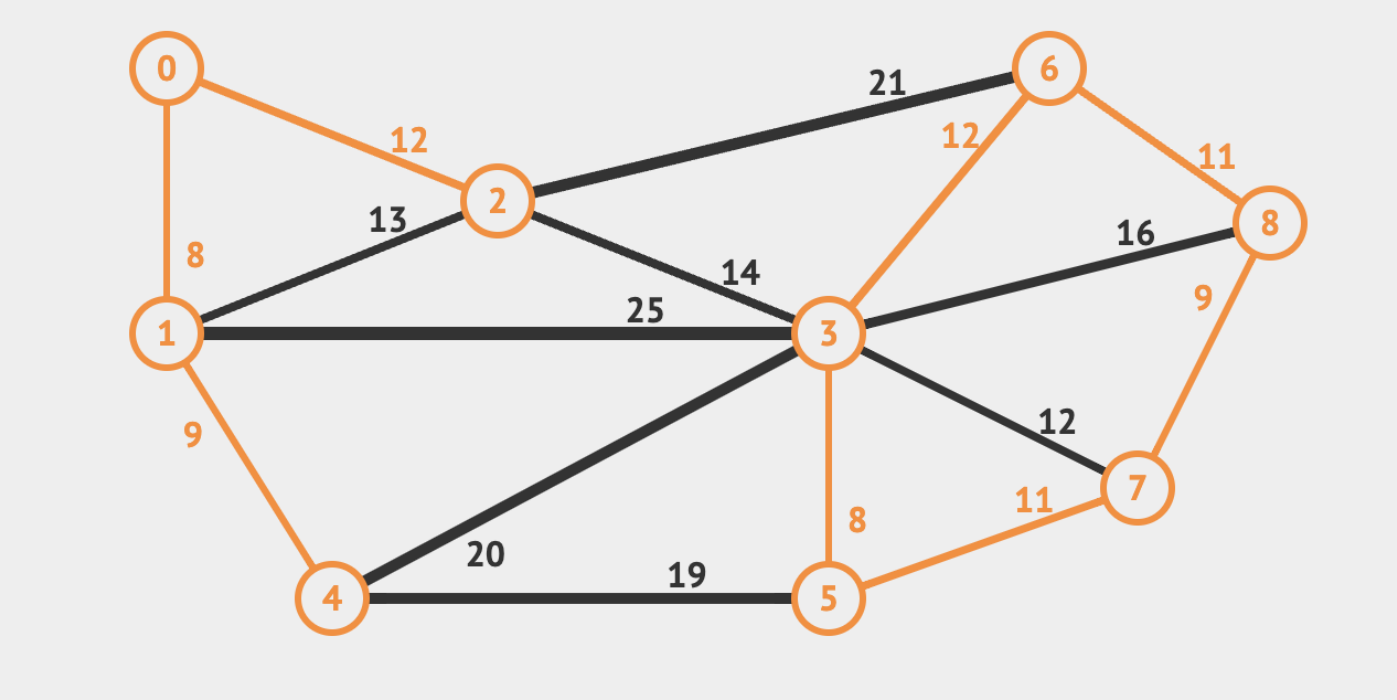
Kruskal's Algorithm Example



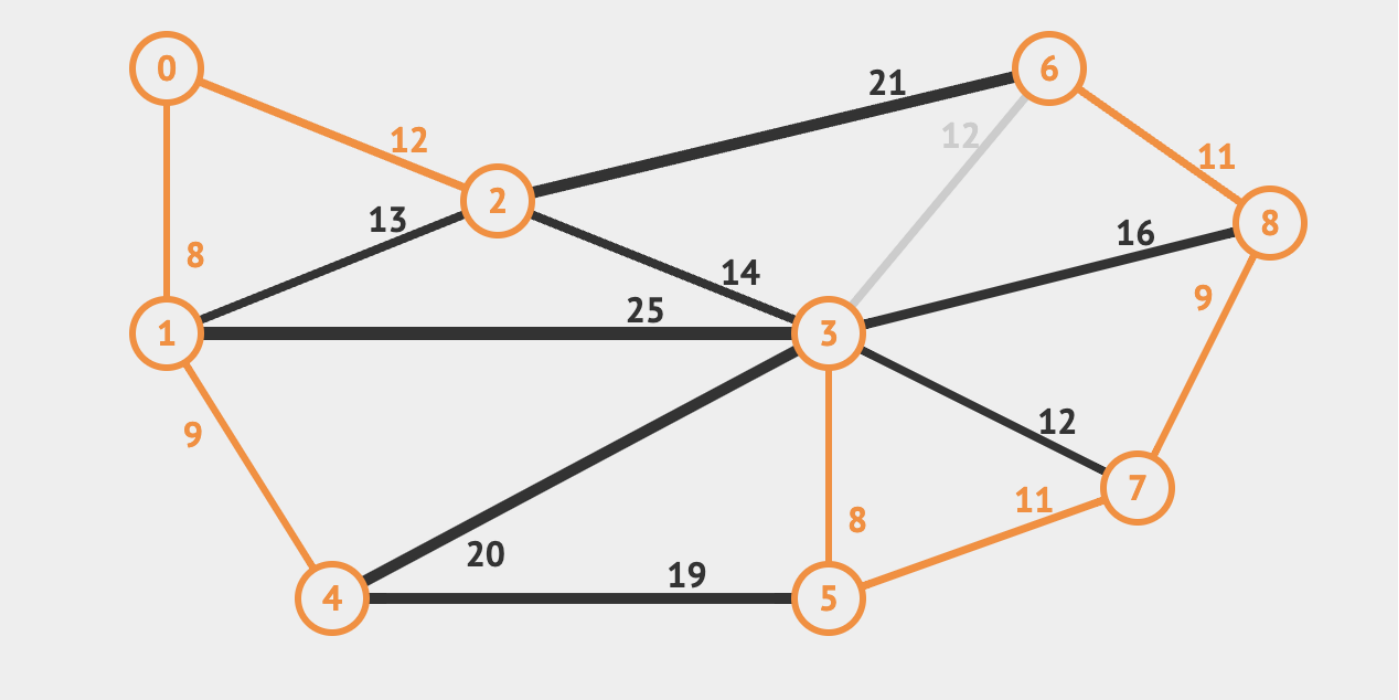
Kruskal's Algorithm Example



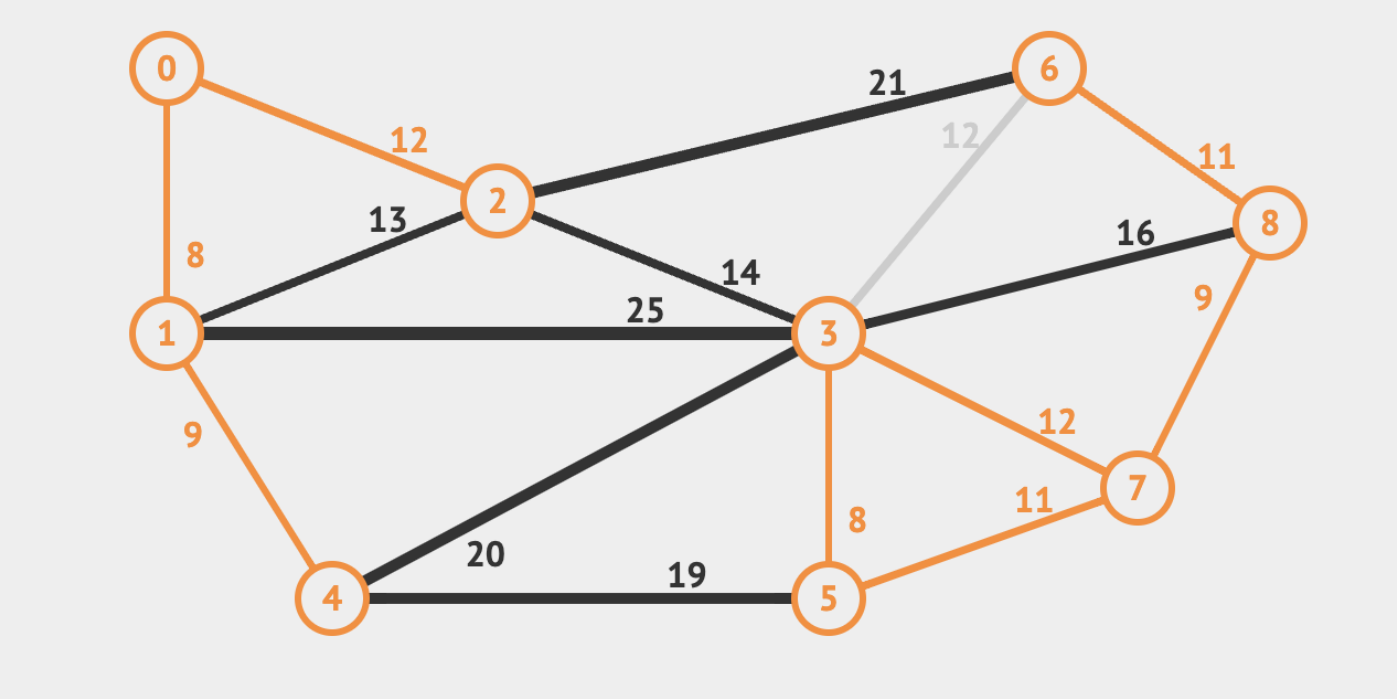
Kruskal's Algorithm Example



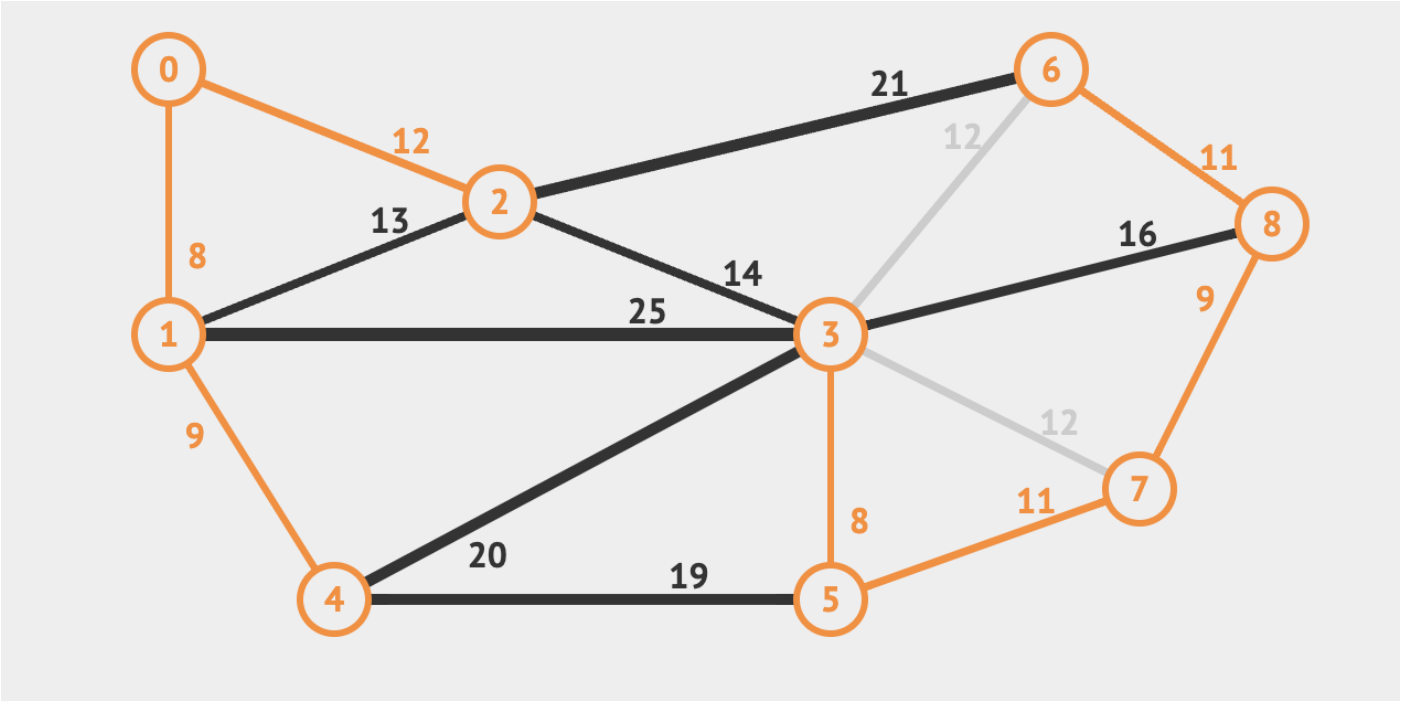
Kruskal's Algorithm Example



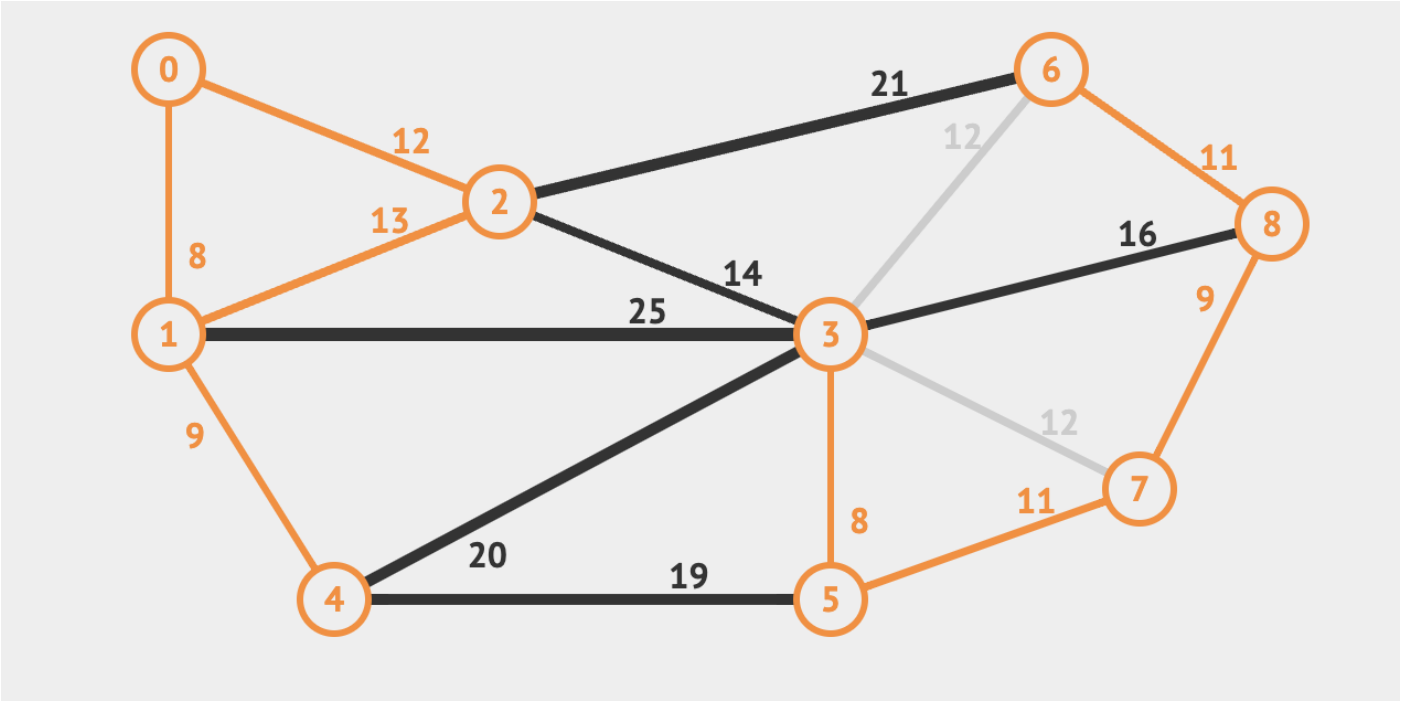
Kruskal's Algorithm Example



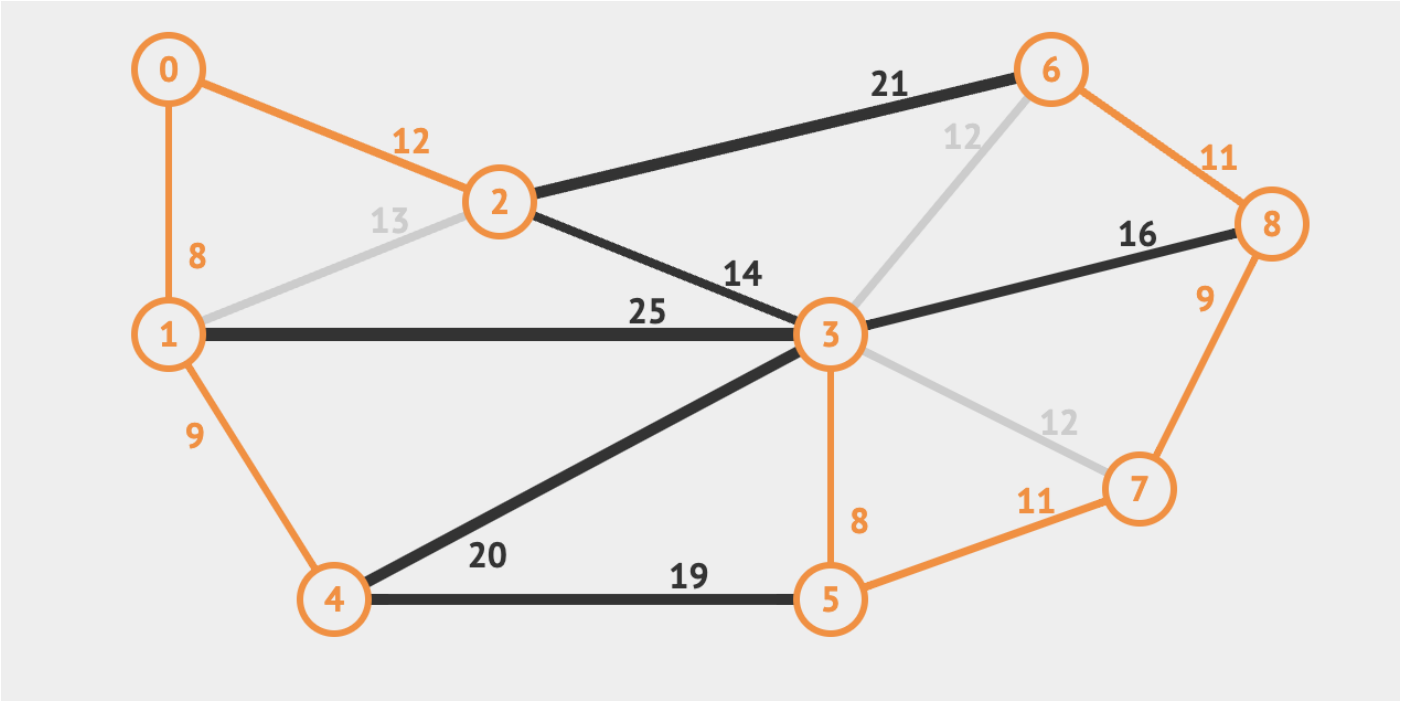
Kruskal's Algorithm Example



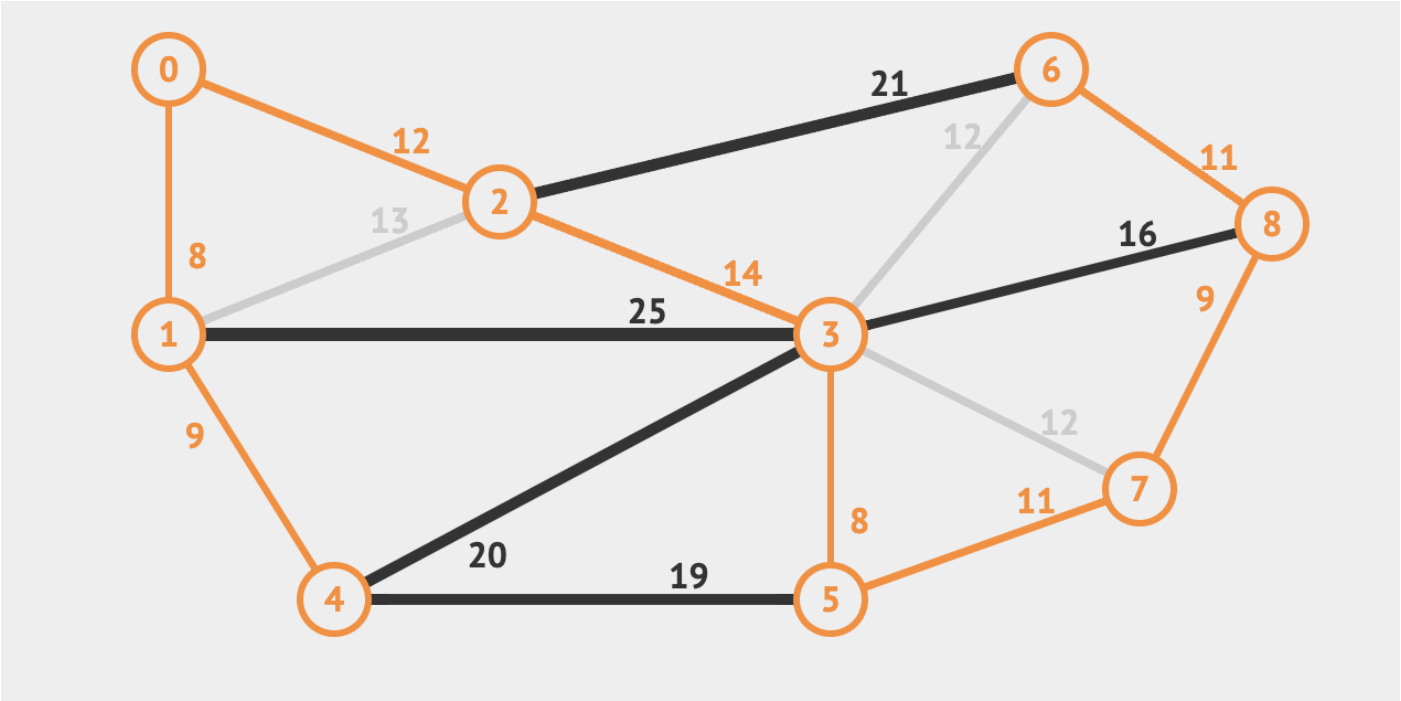
Kruskal's Algorithm Example



Kruskal's Algorithm Example



Kruskal's Algorithm Example



Proof of Correctness for Kruskal's Algorithm

- Let $T = (V, F)$ be the tree produced by Kruskal's algorithm, and let $T^* = (V, F^*)$ be an optimal MST.
- Then, there exists an edge $e \in F^*$ such that $e \notin F$.

Cycle Creation and Edge Replacement

- Adding e to T creates a cycle C.
- In C, there exists an edge $f \in F$ crossing the cut defined by $T^* - e$.
- Since Kruskal's algorithm adds edges in increasing order, e is the most expensive edge in C.
- Thus, $\text{cost}(f) \leq \text{cost}(e)$.

Exchange Step

- Replace e with f in T^* to create a new MST with no greater cost, now sharing more edges with T.

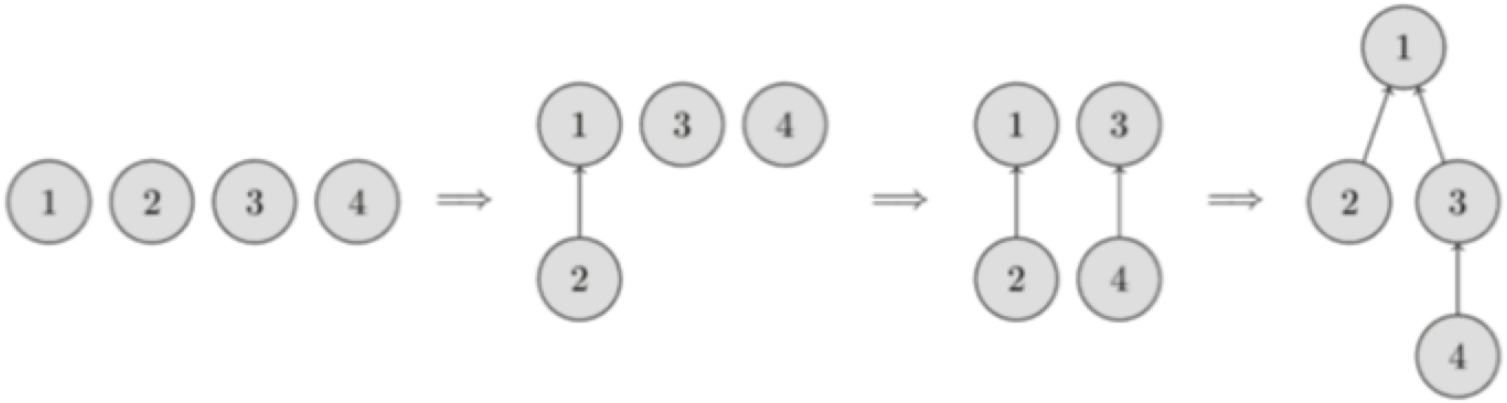
Conclusion

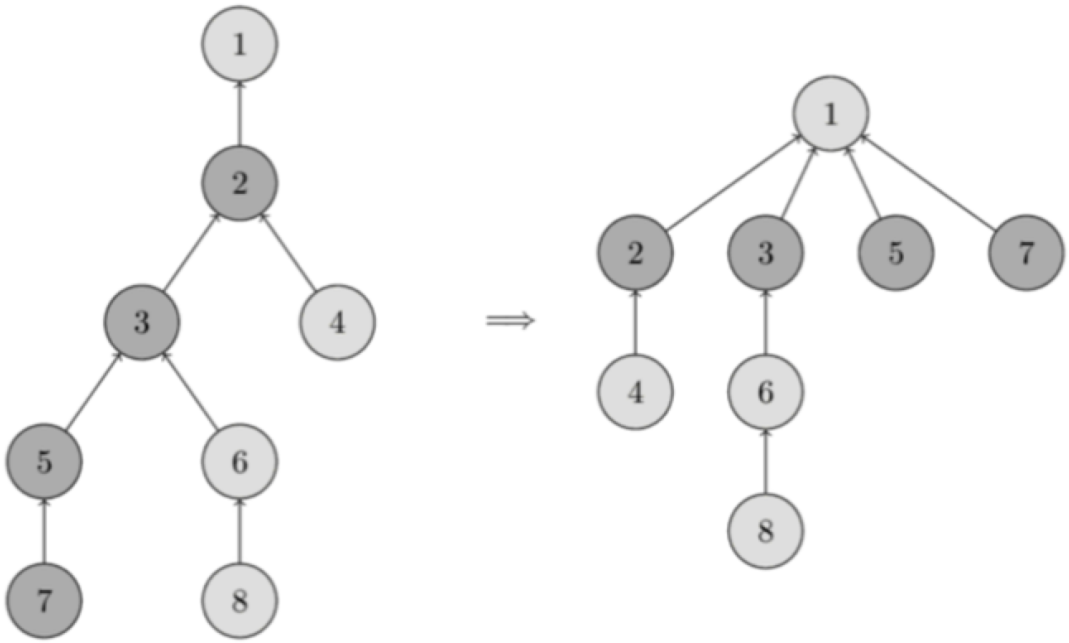
- Repeating this process aligns T with T^*, proving T is an MST.

Union-Find Data Structure

- Maintains disjoint sets and supports two primary operations:
  - `Find(x)`: Determines the set containing x.
  - `Union(x, y)`: Merges the sets containing x and y.
- **Union by Rank** and **Path Compression** optimize performance.
  - Union by Rank: Merges by attaching the root of the smaller tree under the root of the larger tree.
  - Path Compression: Flattens the tree structure, speeding up future find operations.

Union-Find Data Structure





Kruskal's Algorithm with Union-Find

- Initialize $F = \emptyset$ and create a Union-Find structure for all vertices.
- Sort edges of G by weight.
- For each edge $e = (u, v)$ in sorted order:
  - If u and v are in different sets (no cycle), add e to F.
  - Union the sets containing u and v.
- Return F as the MST.

Runtime of Kruskal's Algorithm

- Sorting edges takes $O(m \log m)$.
- Each union and find operation takes amortized time of $O(\alpha(m))$.
- The union find operations take $m\alpha(m)$.
- Total running is therefore $O(m \log m)$.

Prim's Algorithm for MST

"Find the cheapest edge to connect a vertex to S"

- Prim's algorithm grows a single connected component.
- Start from any vertex v and add the minimum-cost edge connecting the component to a new vertex.
- Repeat until all vertices are included.

Prim's Algorithm

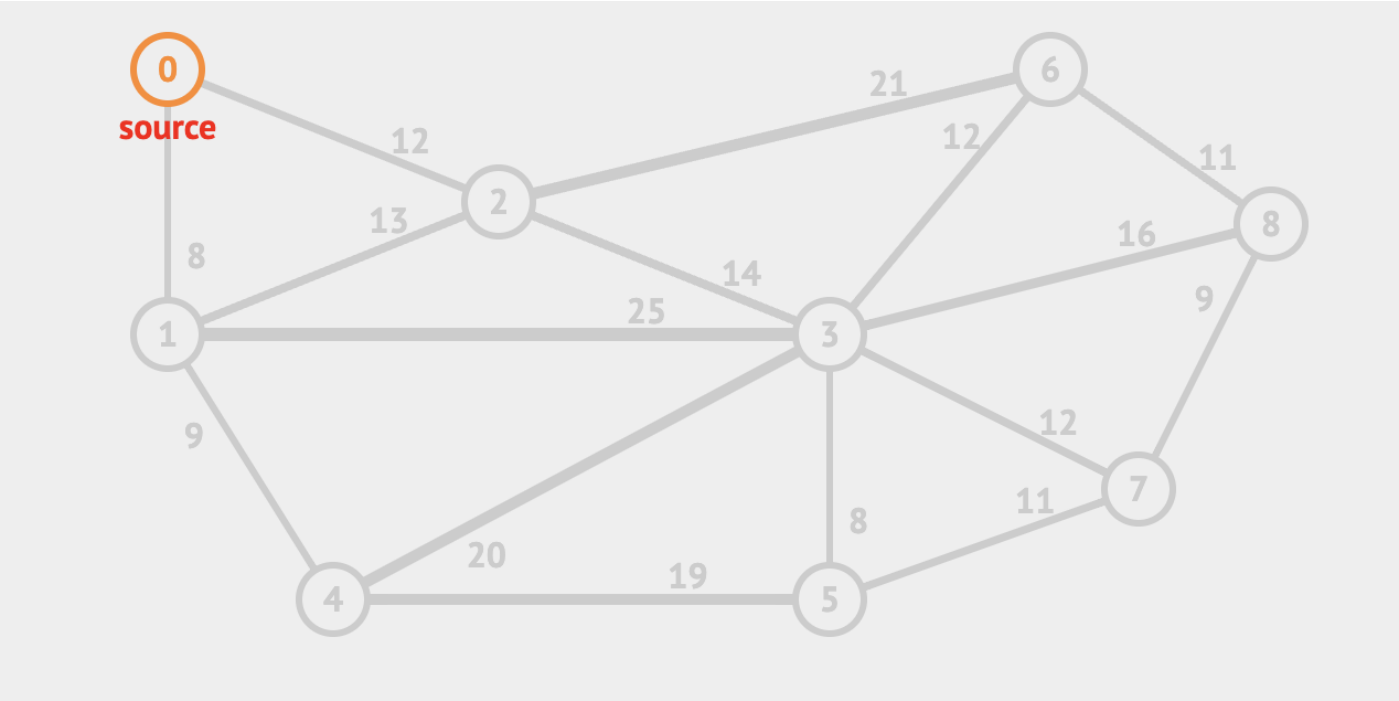
- Prim's algorithm uses a **greedy strategy**, selecting the smallest available edge at each step.
- It ensures the MST is grown by adding one vertex at a time, maintaining connectivity without cycles.
- **Cut Property**: For each step, edges crossing the cut between vertices in the MST and those outside must be minimum-weight.
- The algorithm terminates when all vertices are included, and F is an MST.

Prim's Algorithm: Formal Steps

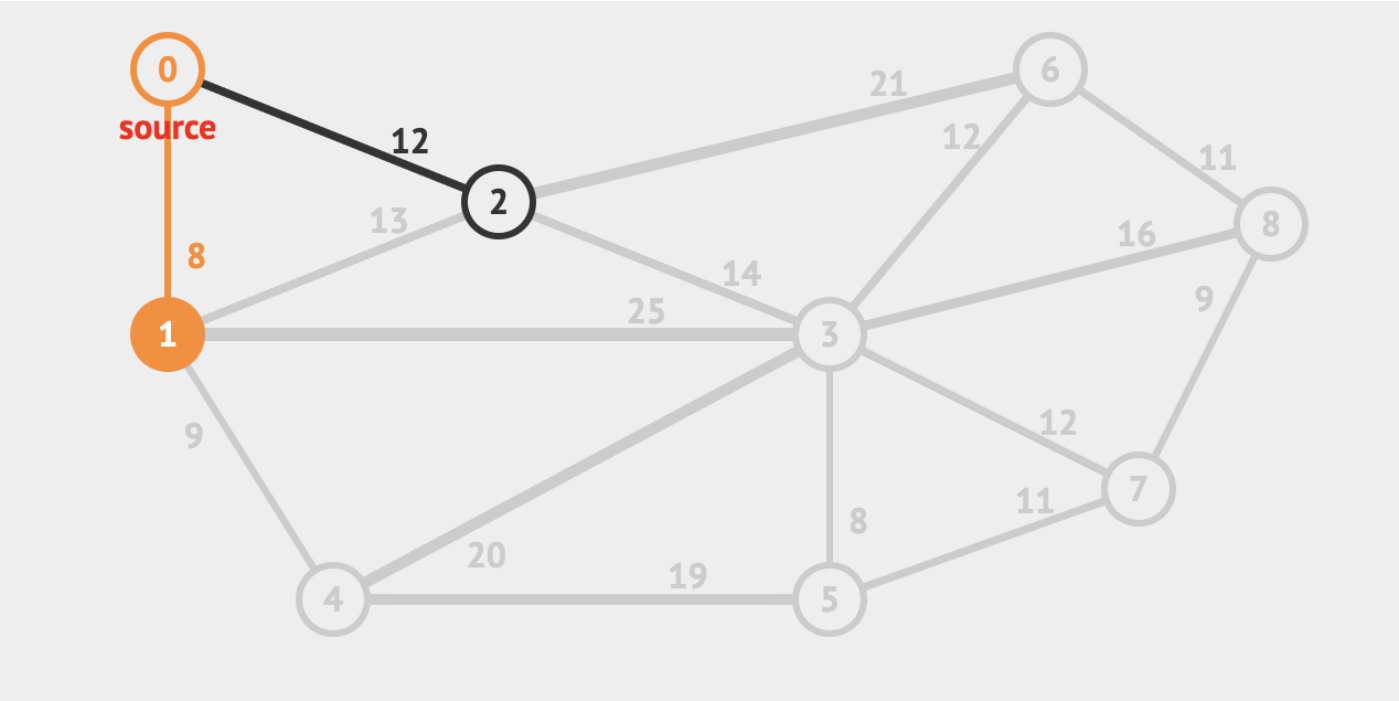
Input: A connected, undirected, weighted graph $G = (V, E)$ with weights $w(e)$ on edges $e \in E$.

1. Initialize an array mark$[1 \ldots n]$ = FALSE and choose any vertex s in V to start.
2. Create an empty forest $F = \emptyset$.
3. Set mark$[s]$ = TRUE and let S be the set of edges incident to s.
4. **While** S is non-empty:
   - Choose the minimum-weight edge $e = \{u, v\}$ in S and remove it from S.
   - **If** both u and v are marked, continue to the next iteration (skip this edge).
   - **Else**:
     - Without loss of generality, assume mark$[v]$ = FALSE (if u is unmarked, switch u and v).
     - Set mark$[v]$ = TRUE.
     - Add $\{u, v\}$ to F and add all edges incident on v to S.
5. Return F as the MST.

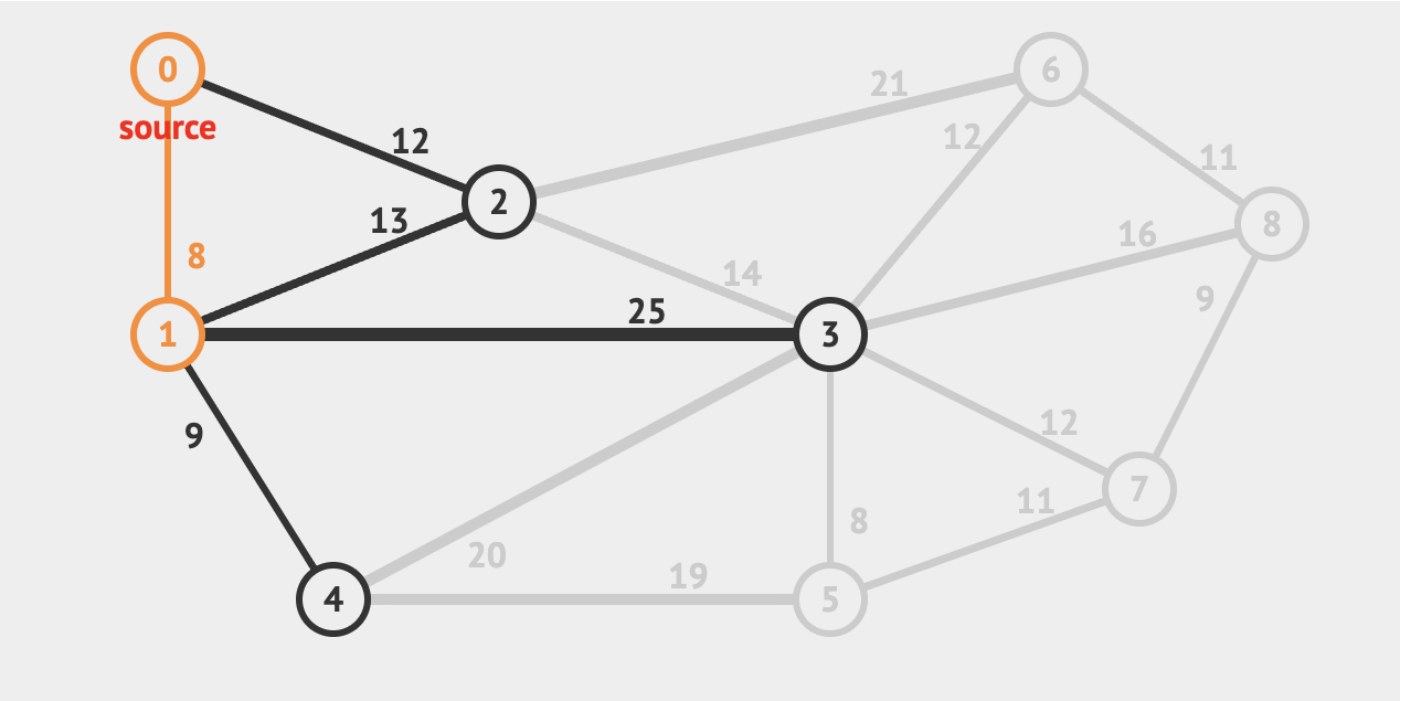
Prim's Algorithm Example



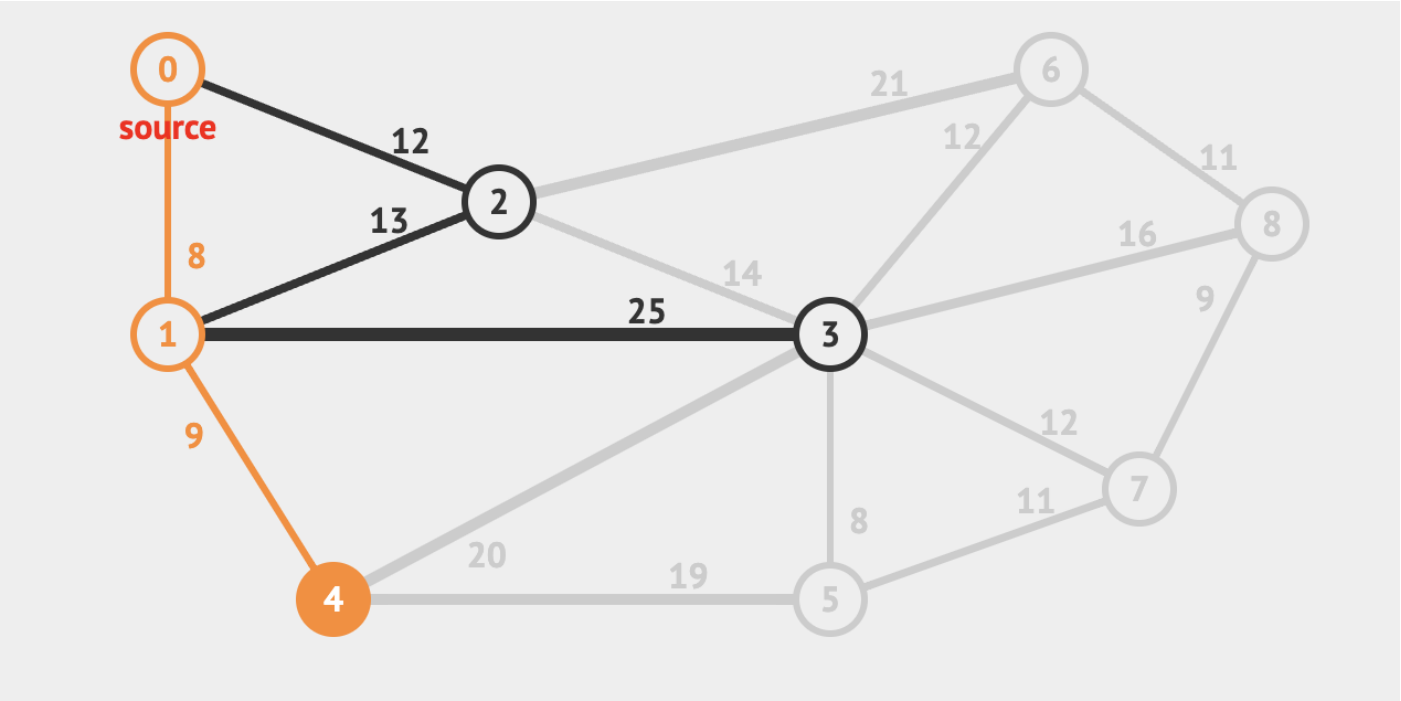
Prim's Algorithm Example



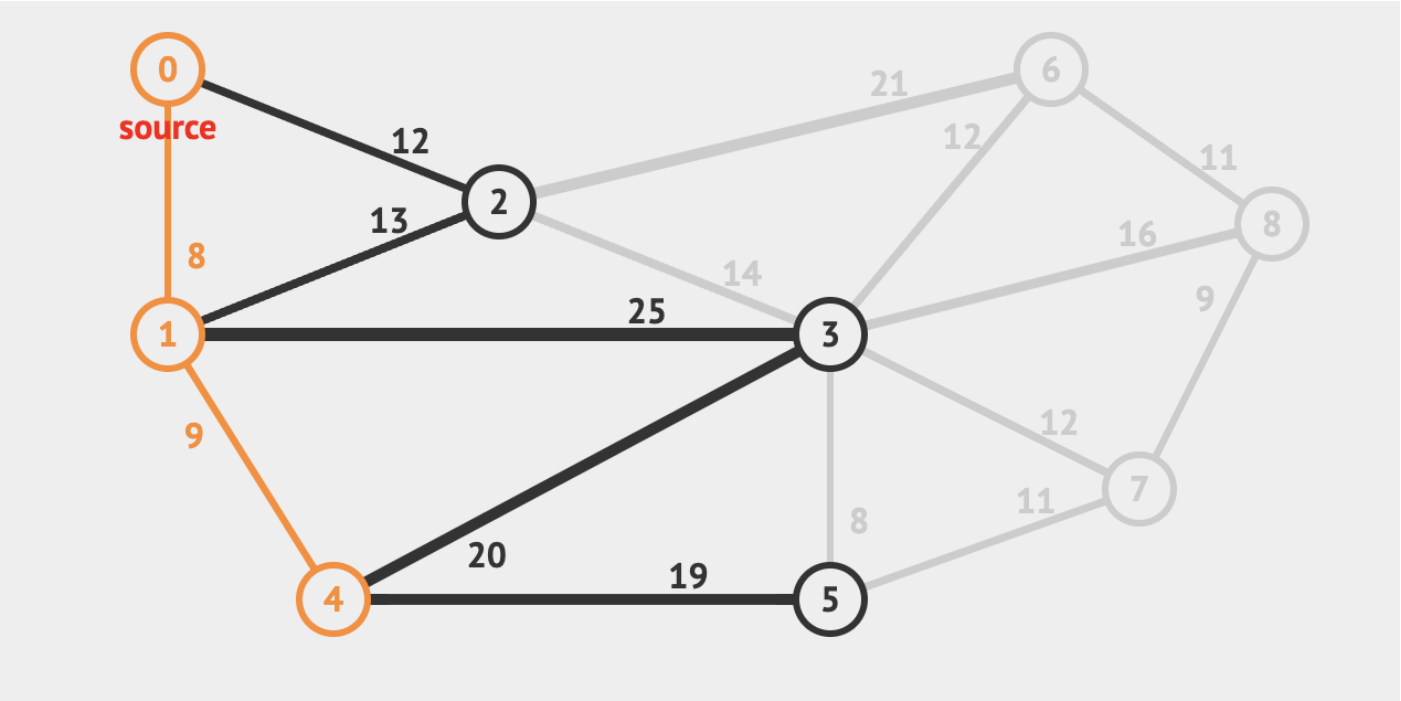
Prim's Algorithm Example



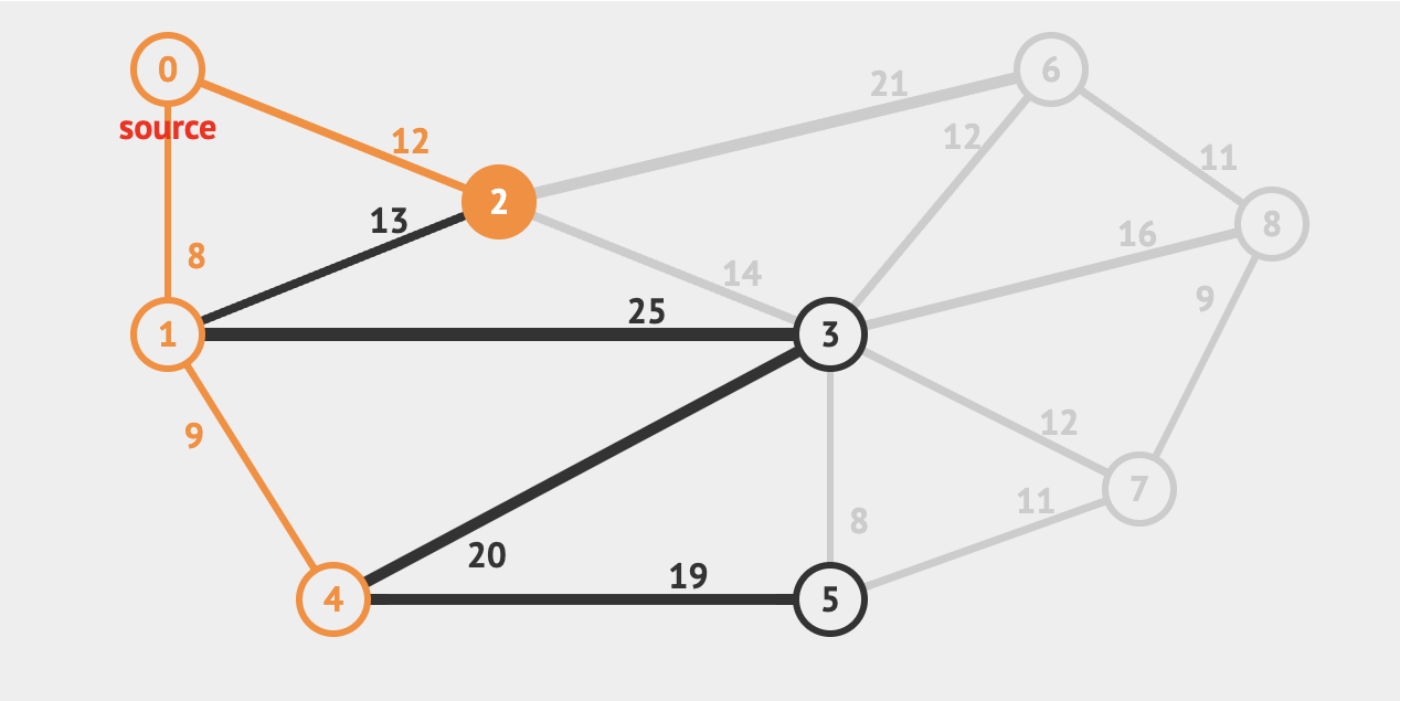
Prim's Algorithm Example



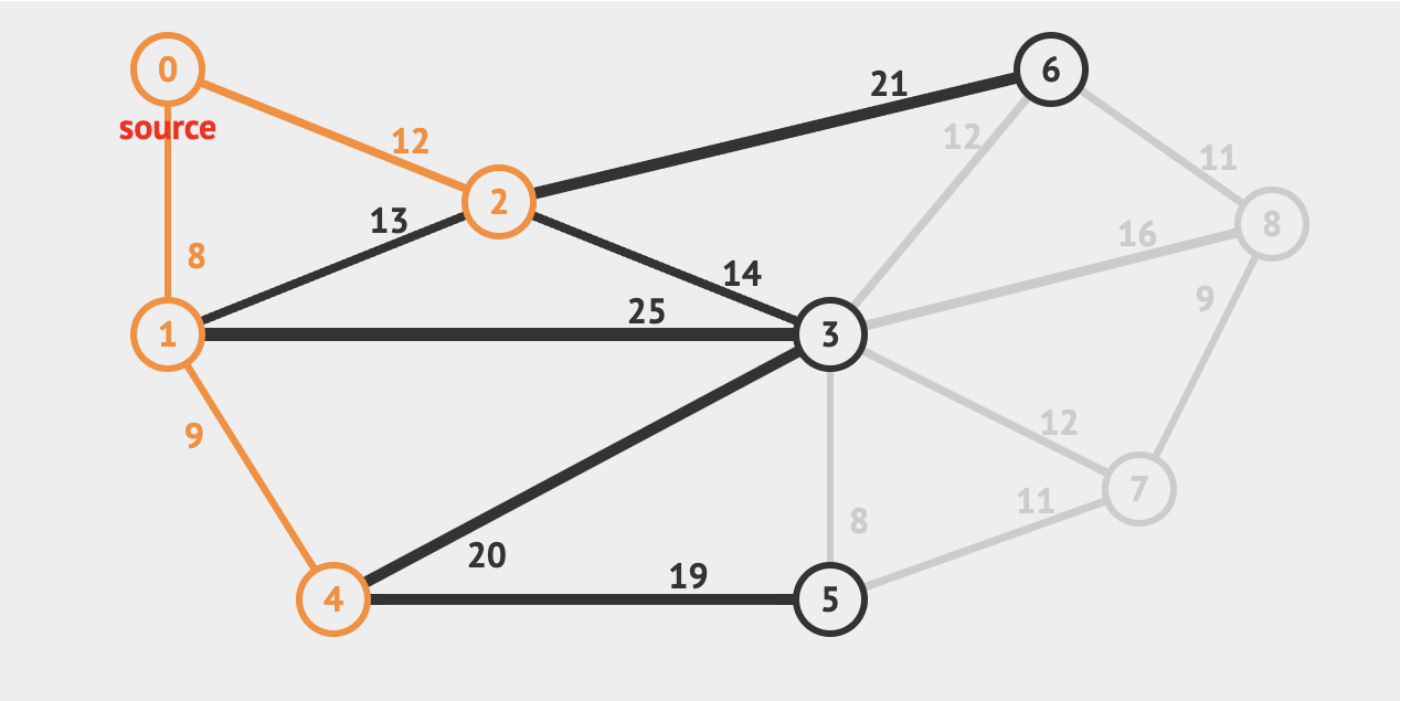
Prim's Algorithm Example



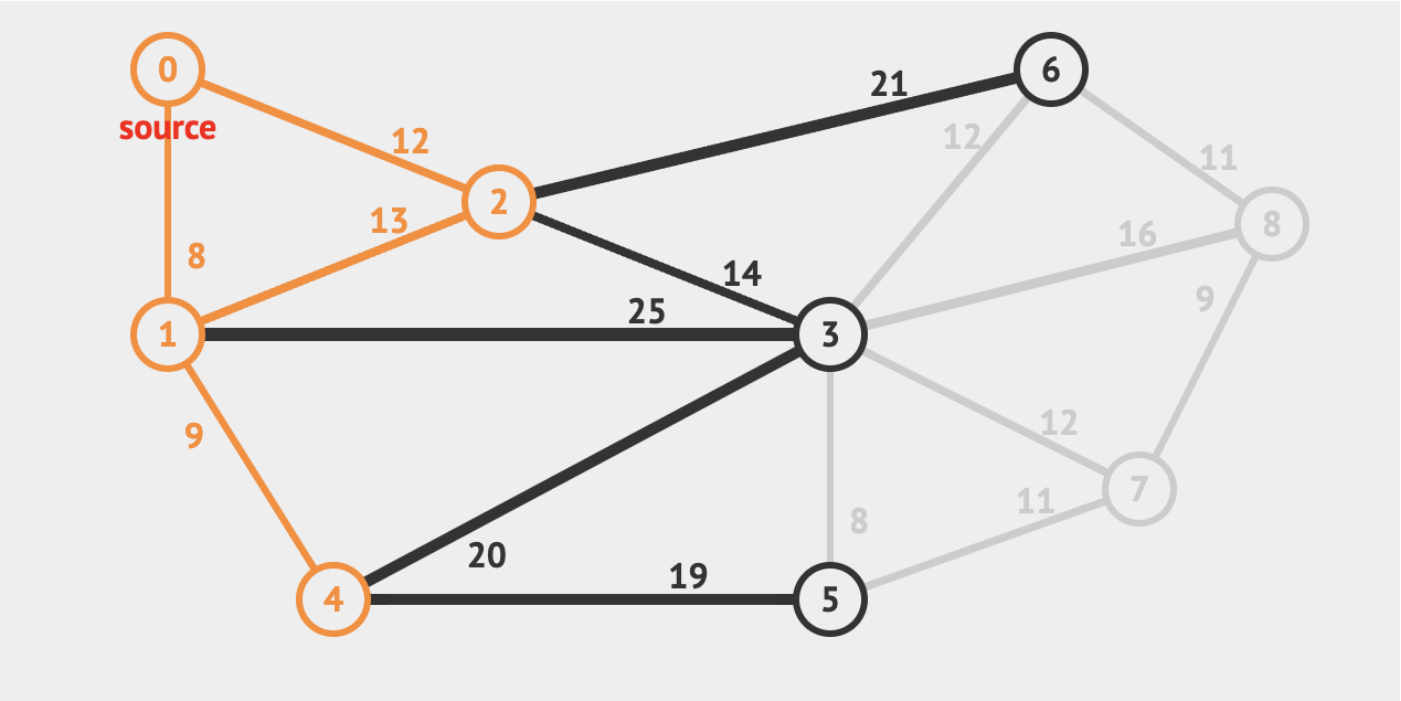
Prim's Algorithm Example



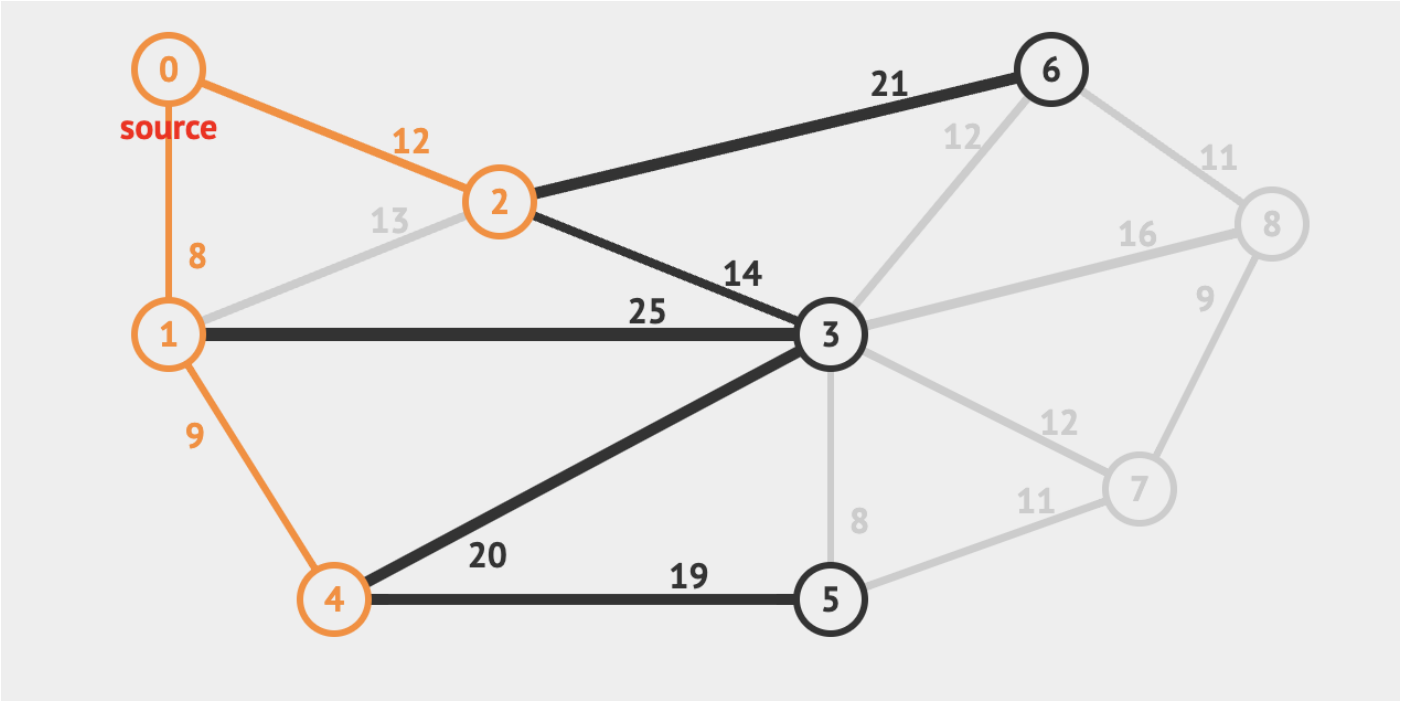
Prim's Algorithm Example



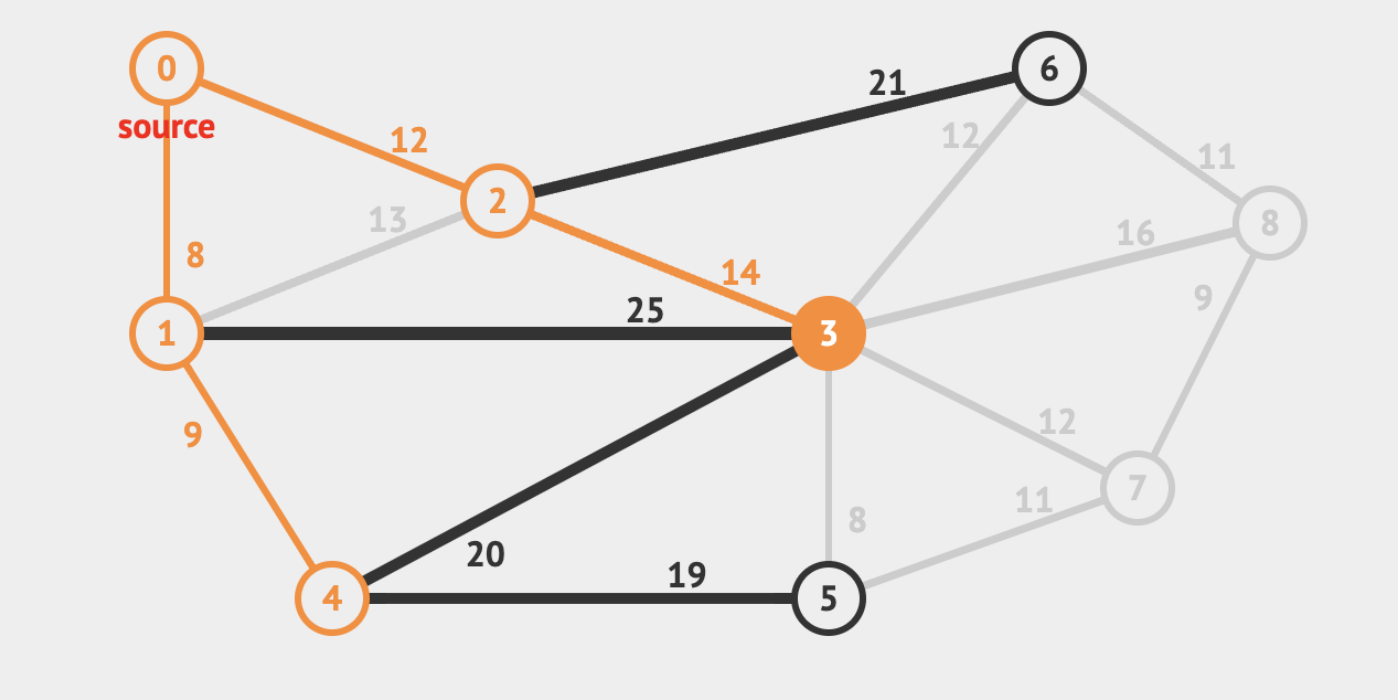
Prim's Algorithm Example



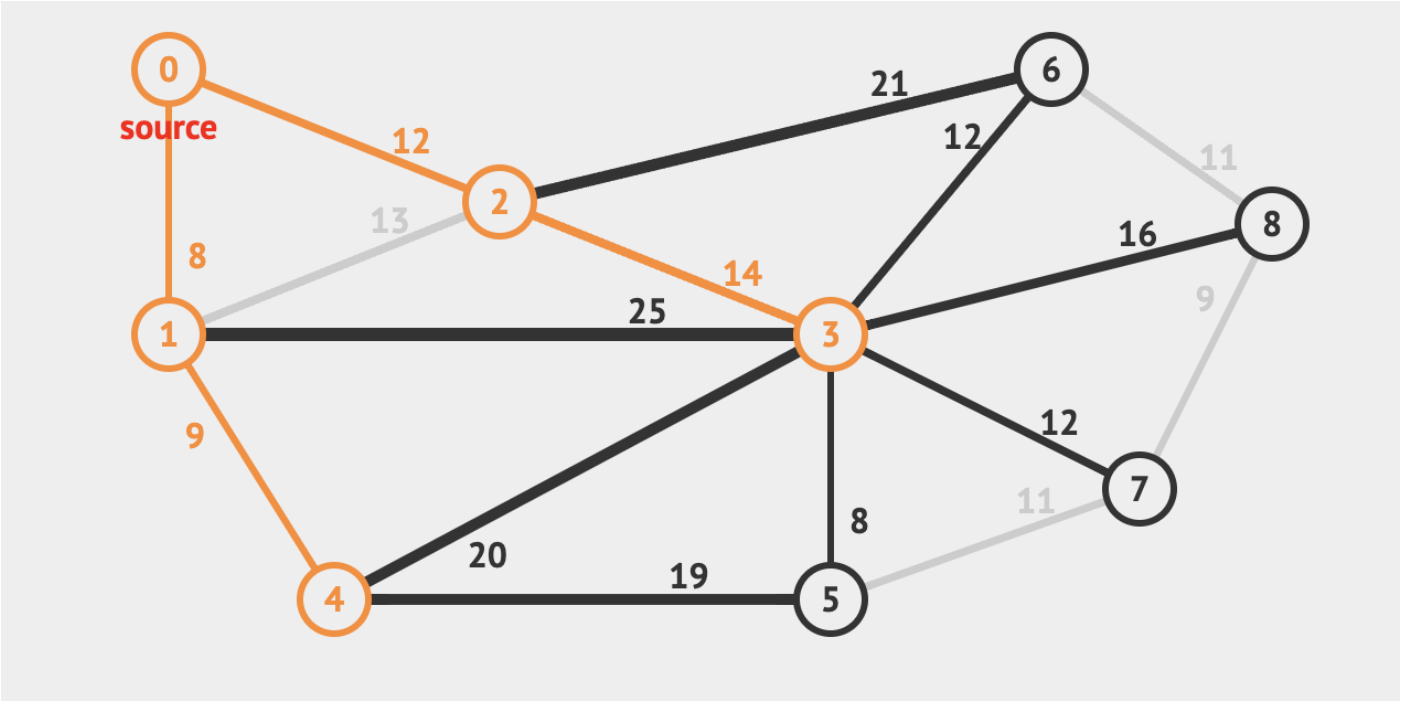
Prim's Algorithm Example



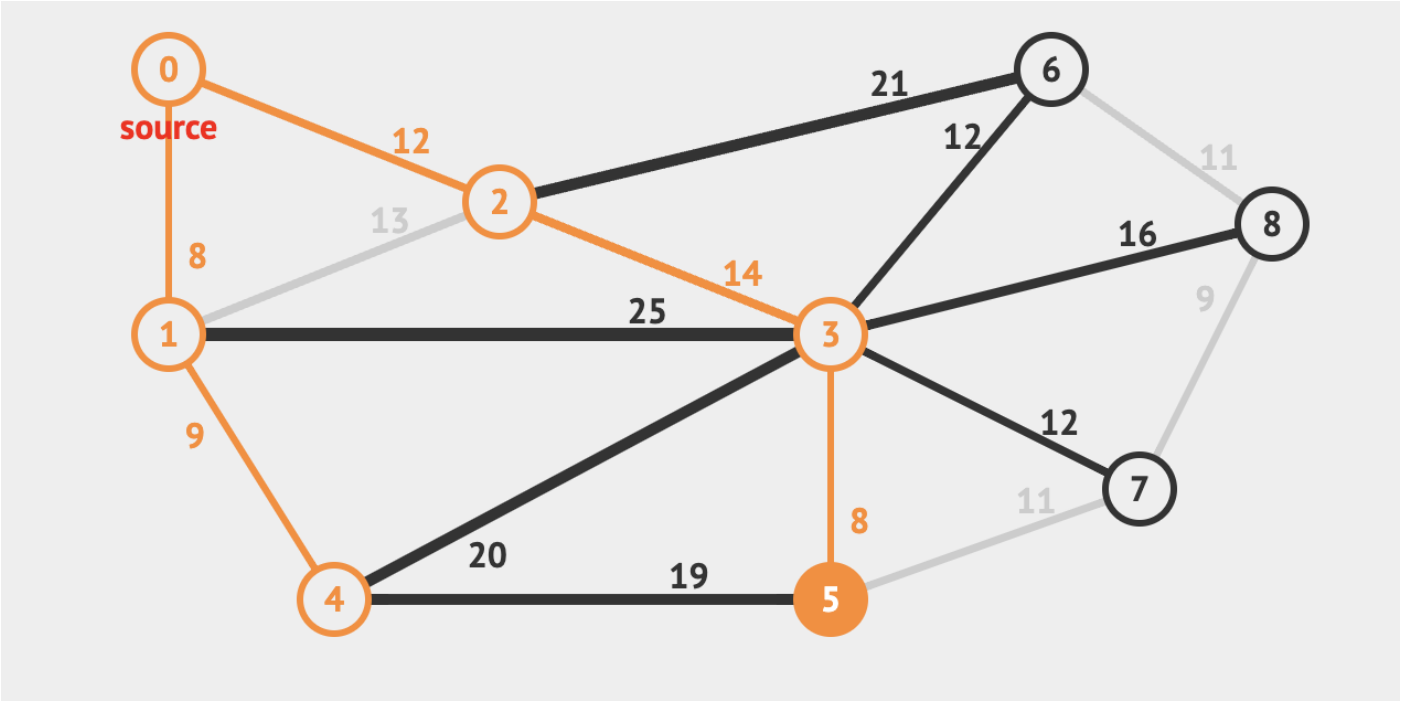
Prim's Algorithm Example



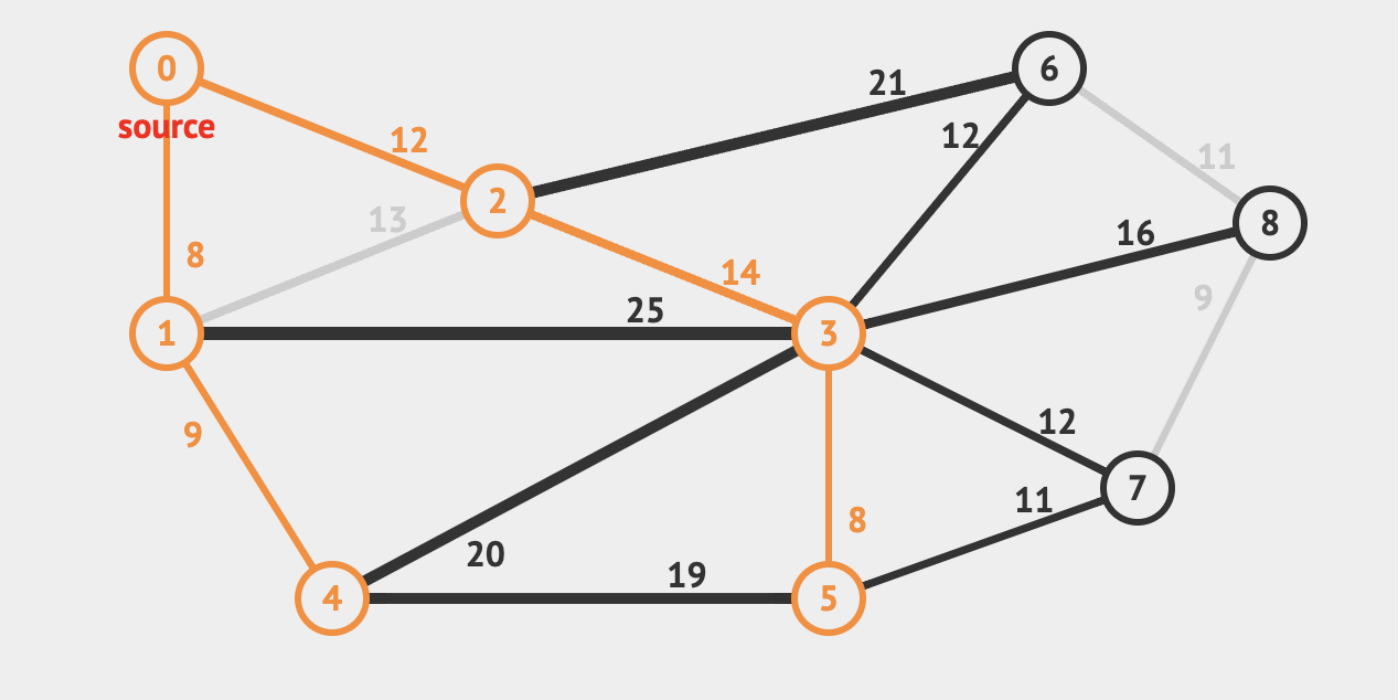
Prim's Algorithm Example



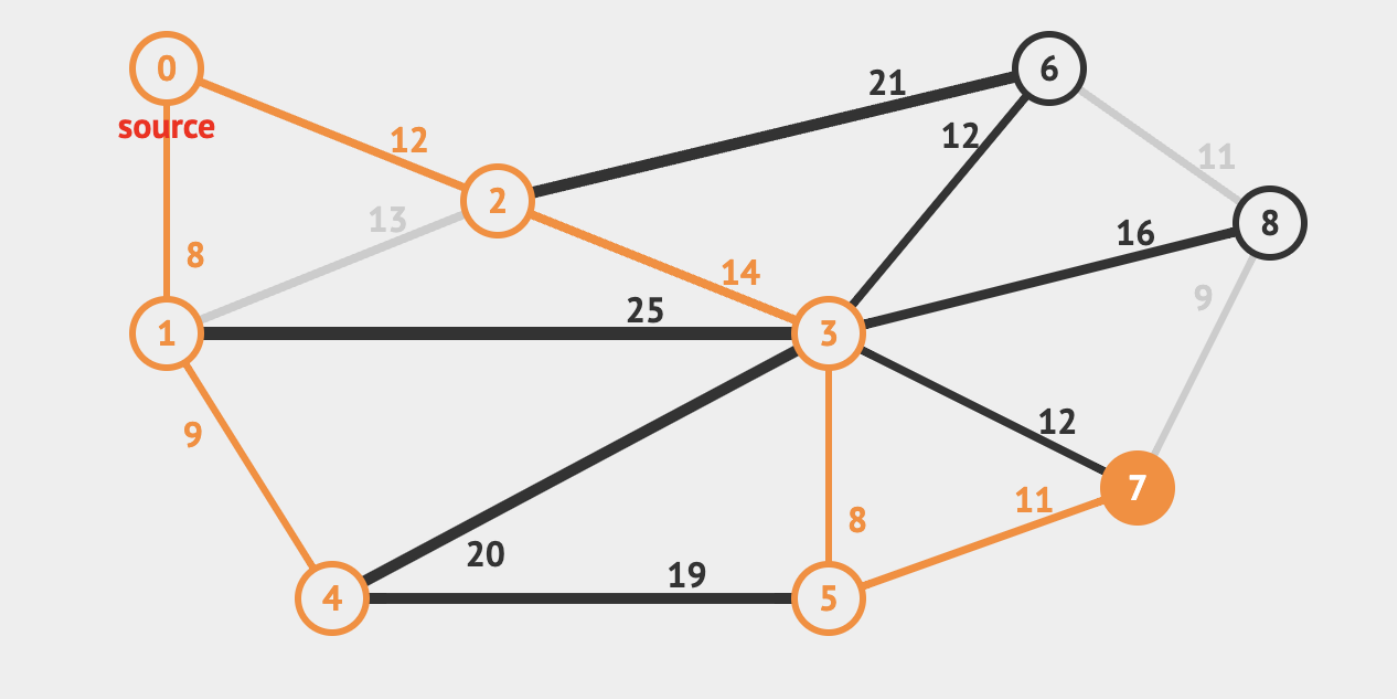
Prim's Algorithm Example



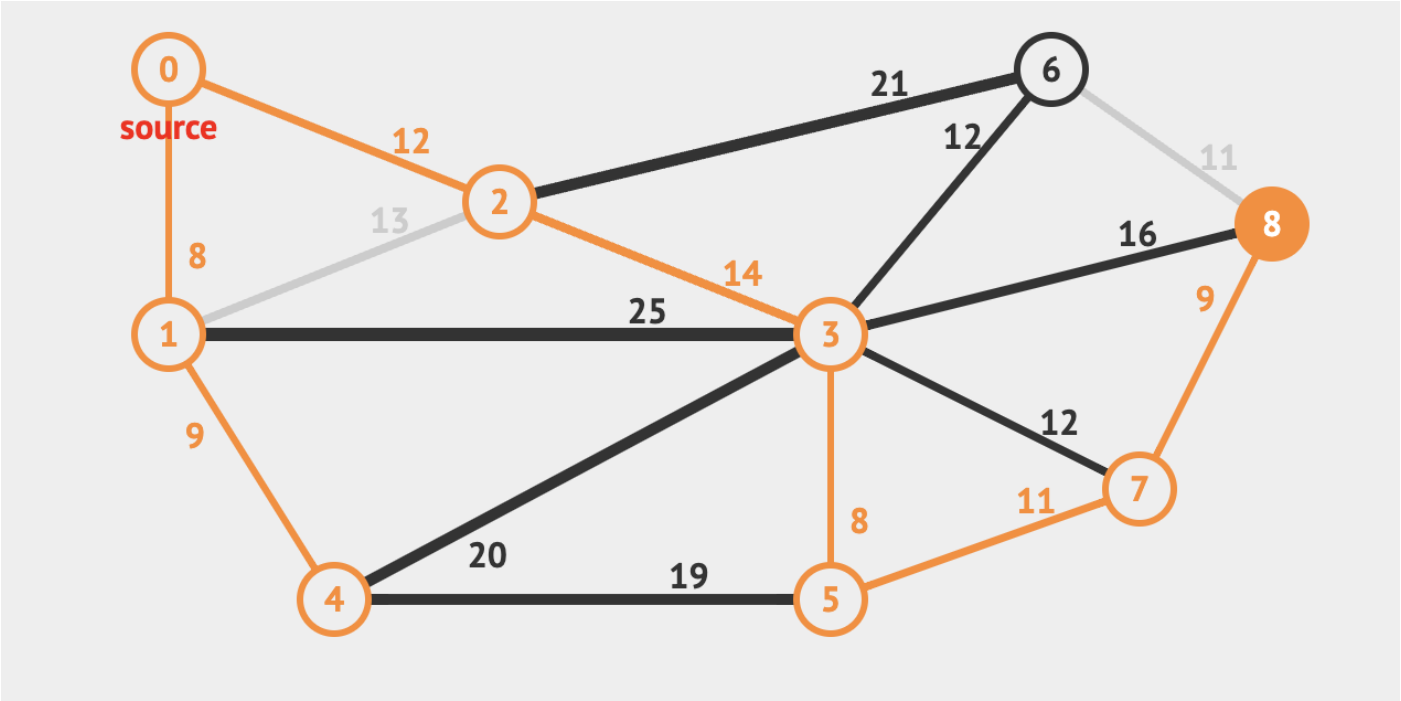
Prim's Algorithm Example



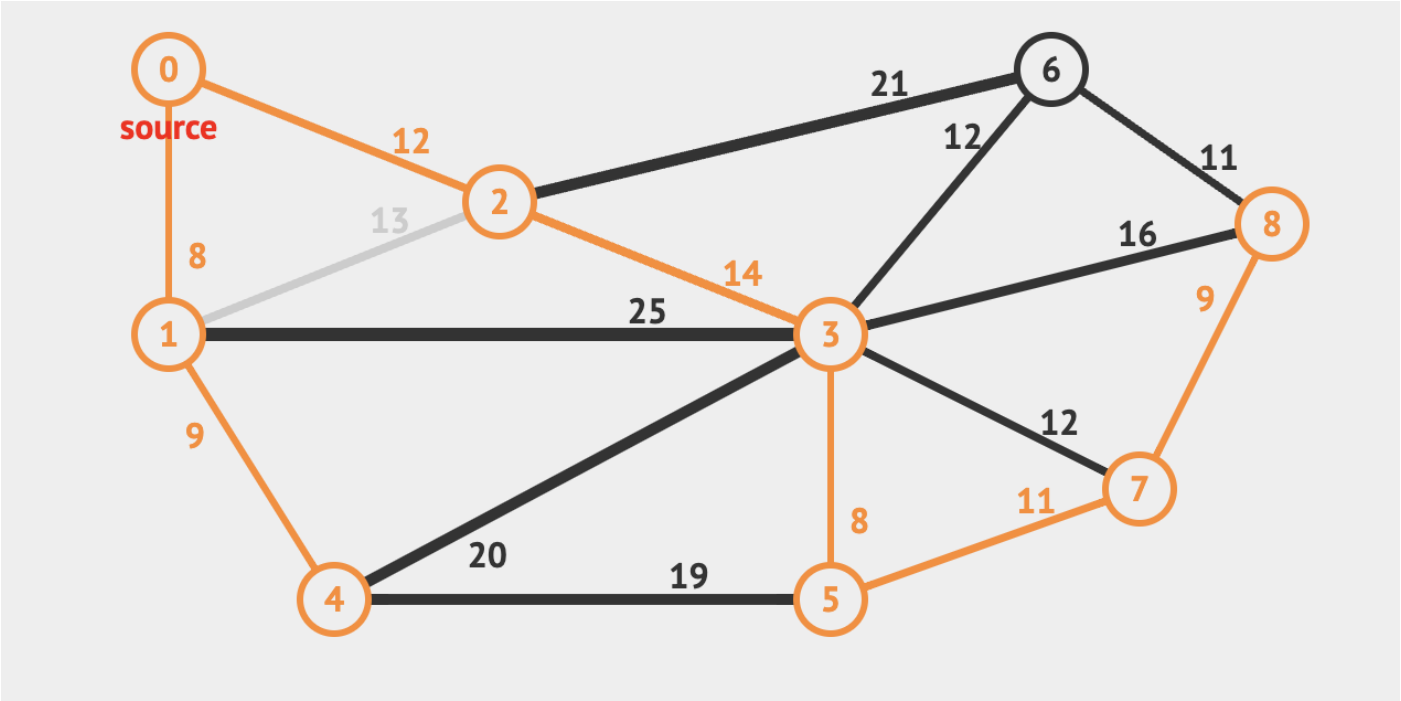
Prim's Algorithm Example



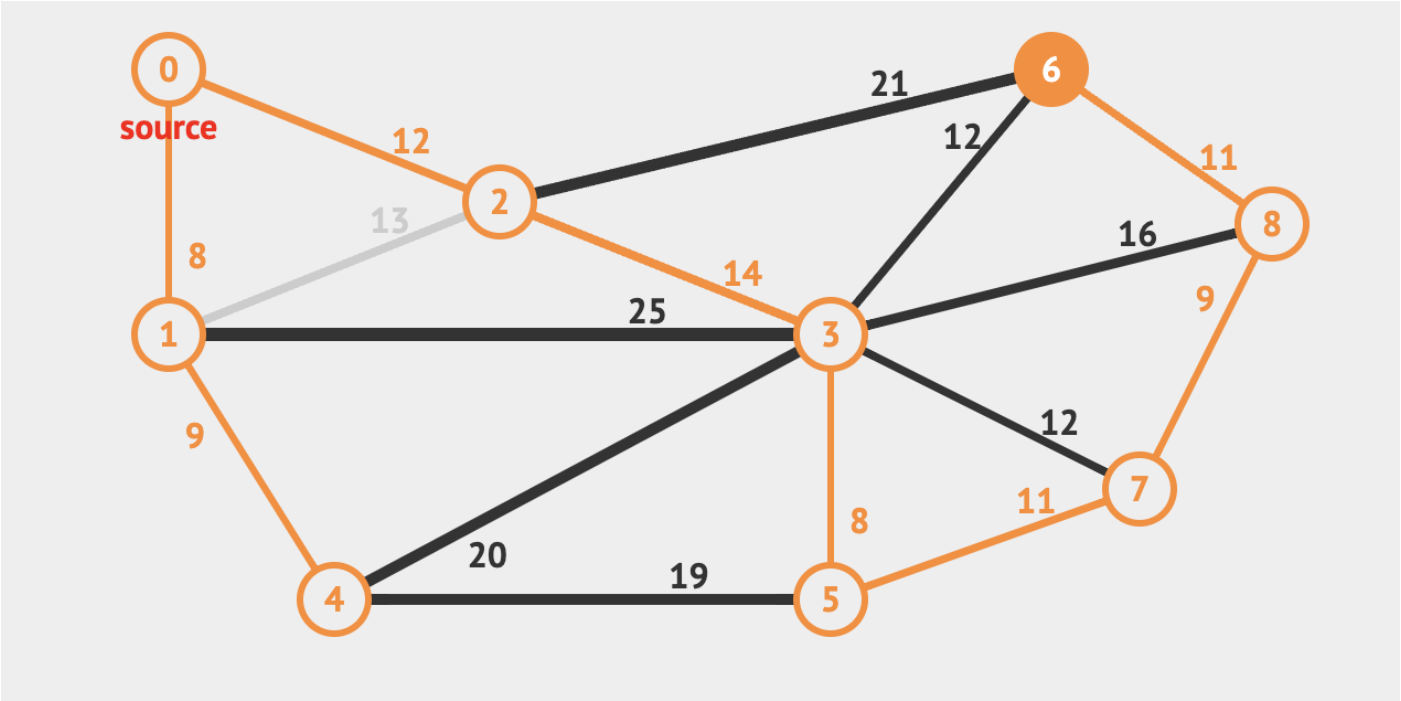
Prim's Algorithm Example



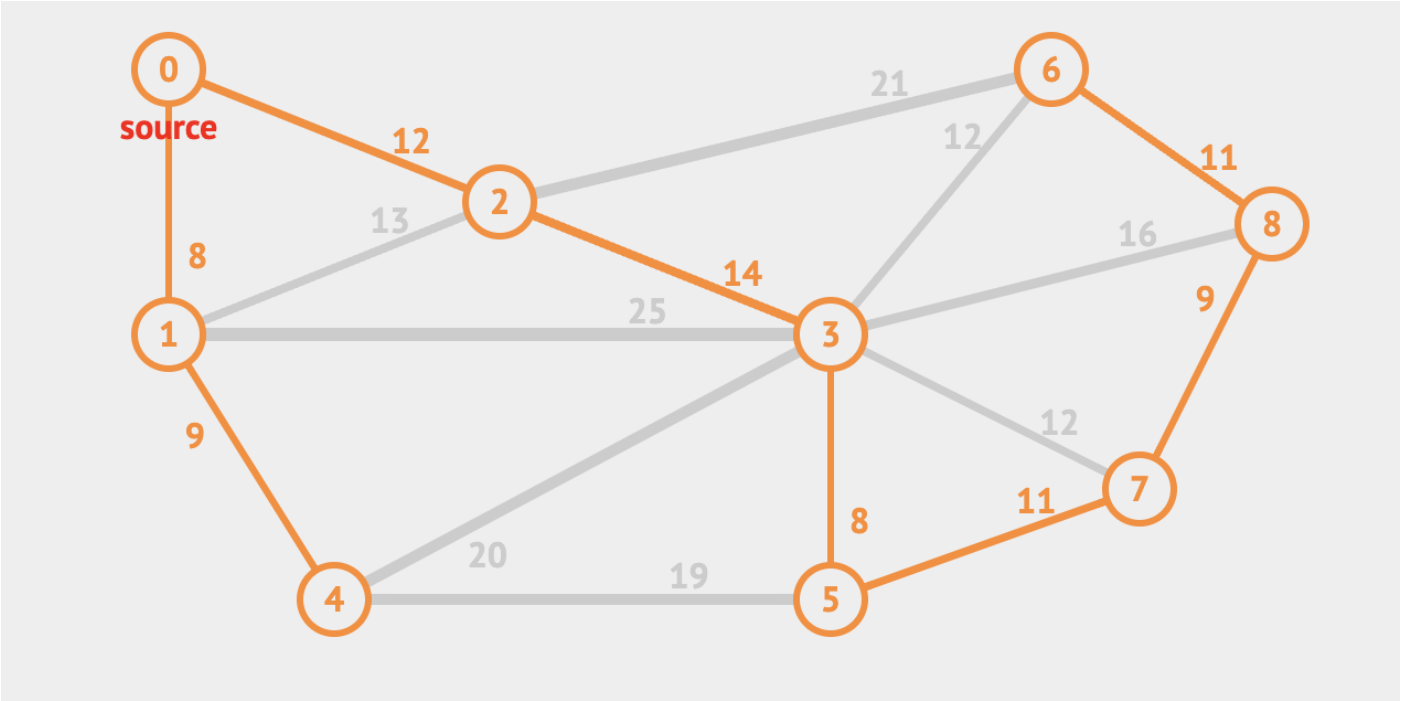
Prim's Algorithm Example



Prim's Algorithm Example



Prim's Algorithm Example



Prim's Algorithm with Min-Heap of Edges

- **Approach**: Use a min-heap to manage edges crossing the MST cut.
- **Algorithm**:
  - Add all edges from a newly added vertex to the heap.
  - Repeatedly extract the smallest edge; if it leads to a new vertex, add it to the MST.
- **Heap Details**:
  - Stores edges; up to $O(E)$ in size.
  - `insert`: $O(\log E)$ per edge.
  - `extract-min`: $O(\log E)$ per edge.
  - **Overall Runtime**: $O(E \log E) = O(E \log V)$.

Algorithm with $d_S[v]$

- The algorithm uses a priority queue Q to store vertices, prioritized by their distance, $\texttt{dist}[v]$, to the growing tree S.
- **Initialization**:
  - Set $\texttt{dist}[\text{start}] = 0$ and all other distances to ∞.
  - Add all vertices to the priority queue Q.
- **Execution Loop**:
  1. Extract the vertex u with the minimum distance from Q.
  2. For each neighbor v of u, if the edge (u, v) provides a cheaper connection to the tree:
     - Update $\texttt{dist}[v]$ to the new, lower weight.
       $\texttt{dist}[v] \leftarrow \min\{\texttt{dist}[v], \text{cost}(u - v)\}$
     - Use `decrease-key` to update v's priority in Q.

Explicit Operation Counts

Here's the breakdown of how many times each key operation is performed during the algorithm's execution.

`extract-min`

- This operation is performed exactly V **times**.
- The algorithm must extract each vertex from the priority queue once to add it to the final MST.

`decrease-key`

- This operation is performed at most E **times**.
- In the worst case, every edge (u, v) could lead to a distance update for vertex v, triggering a call to `decrease-key`. The number of calls is therefore bounded by $O(E)$.

Prim's with a Binary Heap

A standard binary heap provides general-purpose implementation.

- **Operation Costs**:
  - `extract-min`: $O(\log V)$
  - `decrease-key`: $O(\log V)$

- **Total Runtime**:
  - The total complexity is $V \log V + E \log V = O(E \log V)$.

Prim's with a Fibonacci Heap

For better performance, especially on dense graphs, a Fibonacci heap is used.

- **Operation Costs**:
  - `extract-min`: $O(\log V)$
  - `decrease-key`: $O(1)$ on average

- **Total Runtime**:
  - The total complexity is $V \log V + E(1) = O(E + V \log V)$.

Comparison of Kruskal's and Prim's Algorithms

- **Kruskal's**:
  - Suitable for sparse graphs.
  - Relies on Union-Find structure for efficient cycle detection.
- **Prim's**:
  - Suitable for dense graphs.
  - Uses a priority queue to grow a single connected component.
- Both achieve $O(m \log m)$ in general.

Borůvka's Algorithm for Minimum Spanning Tree (MST)

Borůvka's algorithm finds the MST by repeatedly adding the cheapest edge for each component. The algorithm works by merging components until only one spanning tree remains.

Algorithm Steps:

1. **Initialize**: Start with each vertex in its own component, creating an initial forest F of isolated nodes.
2. **Main Loop**:
   - For each component, find the minimum-weight edge that connects it to a different component.
   - Add these cheapest edges to F, merging components.
   - Update the component count; if F is a single component, the MST is complete.

Borůvka's Algorithm Complexity and Efficiency

Efficient Edge Selection:

- Subroutine `AddAllSafeEdges` identifies the minimum edge for each component in $O(E)$ time, where E is the number of edges.
- Connected component counting runs in $O(V)$, where V is the number of vertices.

Time Complexity:

- Each iteration reduces component count by at least half, so the main loop runs $O(\log V)$ times.
- Each iteration takes $O(E)$, leading to an overall time complexity of $O(E \log V)$.

Applications: Borůvka's algorithm is efficient and well-suited for parallel processing, as minimum edge selection can be done independently for each component.

Optimize Water Distribution in a Village

Problem Statement:

- You are given n houses in a village, each needing water supply.
- Water can be provided in two ways:
  1. Build individual wells for each house, with specified costs.
  2. Connect houses with pipes, with specified costs for each connection.
- Goal: Minimize the total cost to supply water to all houses.

Input:

- `wells` array where `wells[i]` is the cost to build a well for house i.
- `pipes` array where each element $[u, v, cost]$ represents a pipe between houses u and v with a given cost.

Optimize Water Distribution in a Village

Solution:

- Treat each house as a node in a graph.
- Introduce a virtual node connected to each house with an edge (well) weighted by the cost of building a well.
- Add all pipes as edges between houses with their respective costs.
- Use an MST algorithm (e.g., Kruskal's or Prim's) to find the minimum cost to connect all nodes, including the virtual node.

Complexity:

- Graph building: $O(V + E)$, where E is the number of pipes.
- MST computation (Kruskal's or Prim's): $O(E \log E)$.

Rebuild the City

Problem: The city needs to be reconnected with minimum repair costs. Each road has a repair cost, and the goal is to fully reconnect the city at the lowest total expense.

Objective: Minimize the total cost of repairs to keep the city fully connected.

Approach: - Apply Kruskal's or Prim's algorithm to find the Minimum Spanning Tree (MST). - Sort roads by cost (for Kruskal) or use a priority queue (for Prim) to select minimum-cost roads. - Sum the costs of the selected roads to obtain the final answer.

Complexity: $O(E \log E)$, where E is the number of roads.

The Earliest Moment When Everyone Becomes Friends

Determine the earliest time when everyone in a network is connected, given friendship logs `[timestamp, personA, personB]`.

Approach:

1. Sort logs by timestamp.
2. Use Union-Find to track connections between individuals.
3. For each log, `union` the two friends. If only one connected component remains, record the timestamp and stop.

Complexity: Sorting+ Union-Find $O(E \log E)$